

Università Politecnica delle Marche

Facoltà di Ingegneria

Dipartimento di Ingegneria dell'Informazione

Corso di Laurea Magistrale in Ingegneria Informatica e dell'Automazione



New Generation Databases

**ArtGraph: Progettazione, implementazione e analisi di un Knowledge
Graph in Neo4j per il Cultural Heritage**

Estrazione da Wikidata, modellazione property graph e scenari di retrieval su
artisti, opere, movimenti e collezioni

Professori

Prof. Claudia Diamantini
Prof. Emanuele Storti

Studenti

Loris Bottegoni
Leonardo Cambiotti
Valerio Crocetti

Anno Accademico 2025-2026

Indice

1	Introduzione e Obiettivi del Progetto	2
2	Definizione del Dominio e Costruzione dello Scope	4
2.1	Selezione del Campione di Artisti di Partenza (Seed)	5
2.2	Algoritmo di Espansione Semantica	6
2.3	Risoluzione e Normalizzazione Geografica	6
3	Acquisizione e Trasformazione in ArtGraph RDF	8
3.1	Download dei Dump RDF Grezzi	8
3.2	Trasformazione e Riduzione tramite SPARQL CONSTRUCT	9
3.3	Integrazione Semantica e Gestione delle Lingue	12
4	Modellazione e Traduzione in Property Graph (Neo4j)	14
4.1	Mapping formale RDF-to-LPG	14
4.2	Modello V1: Baseline Entity-Centric	15
4.3	Modello V2: Reificazione delle Relazioni	16
4.4	Pipeline di Gestione e Validazione	17
5	Benchmark Prestazionale del Workload	19
5.1	Le 35 Query	19
5.2	Metodologia di Test e Risultati Prestazionali	33
5.3	Analisi dell’Instabilità di Profiling	35
6	Disegno del Benchmark Comparativo RAG vs GraphRAG	36
6.1	Costruzione del Corpus Testuale Allineato	36
6.2	Generazione e Validazione delle Domande di Test	37
6.3	Le 7 Pipeline di Retrieval a Confronto	38
7	Analisi Sperimentale e Risultati RAG vs GraphRAG	41
7.1	Configurazione dei Test e Metriche Utilizzate	41
7.2	Discussione del Trade-Off Vettoriale vs Grafo Semantico	42
7.3	Confronto delle Strategie a Grafo: Query Generation vs Graph Expansion	46
8	Conclusioni e Sviluppi Futuri	48
8.1	Sintesi dei Risultati e Contributi Metodologici	48
8.2	Limitazioni	49
8.3	Prospettive di Ricerca ed Evoluzioni Future	50

1. Introduzione e Obiettivi del Progetto

Il patrimonio storico-artistico (*Cultural Heritage*) costituisce un dominio informativo interconnesso, rappresentabile tramite un insieme di entità e relazioni semantiche. La descrizione di un'opera d'arte, infatti, non può prescindere dalla rete di legami che la unisce, ad esempio, all'autore, ai suoi maestri, alle correnti stilistiche di riferimento, ai materiali impiegati e al museo in cui è esposta. Attualmente, l'accesso a questa conoscenza può avvenire tramite due principali modalità, entrambe presentanti limiti significativi:

- **Corpora testuali non strutturati:** Le raccolte di documenti offrono un elevato livello di dettaglio descrittivo, ma risultano difficili da interrogare in modo granulare o quantitativo. L'impiego dei Large Language Model (LLM) su tali fonti resta soggetto a fenomeni di allucinazione, e il recupero di passaggi pertinenti da fornire al modello [1] attenua il problema senza risolverlo. La difficoltà cresce quando la risposta non risiede in un singolo passaggio ma va ricomposta lungo una catena di documenti distinti, compito per il quale sono stati costruiti benchmark dedicati [2].
- **Database Relazionali (RDBMS):** I sistemi basati sul modello relazionale impongono schemi rigidi che mal si adattano al dominio artistico. La modellazione di influenze stilistiche pluridecennali o di passaggi di proprietà complessi richiede operazioni computazionalmente onerose.

In questo contesto, i Knowledge Graph (KG) offrono una struttura flessibile e scalabile basata sulla rappresentazione esplicita delle connessioni, e il loro impiego come sorgente di evidenze per i modelli generativi costituisce una risposta diretta ai limiti sopra descritti. Con *GraphRAG* si indicano qui le pipeline in cui l'evidenza fornita al modello proviene da una rappresentazione a grafo. Il lavoro parte da un grafo già esistente, ricavato da Wikidata, e ne esplora l'impiego come sorgente di evidenze: una direzione complementare a quella di Edge et al. [3], dove il grafo viene invece costruito a partire dai documenti.

Nel dettaglio, questo progetto si compone di 4 attività principali:

1. **Costruzione del grafo:** Definire un flusso di lavoro riproducibile e controllato per l'estrazione, la riduzione e il grounding dei dati estratti da Wikidata.
2. **Sfida tra modelli di modellazione:** Progettare e implementare in Neo4j due diverse strategie di modellazione logica. La prima (V1) adotta una topologia basata su relazioni dirette tra le entità; la seconda (V2) prevede la reificazione selettiva delle relazioni più ricche mediante nodi intermedi, allo scopo di analizzare l'impatto di tale scelta in termini di overhead strutturale di storage, complessità di scrittura ed efficienza di scansione delle query Cypher.
3. **Benchmark dei sistemi:** Misurare le prestazioni dei modelli LPG (Label Property Graph) V1 e V2 e del modello RDF (tramite *Oxigraph*) su un gruppo di 35 query.
4. **Verifica delle architetture di recupero informativo (RAG vs GraphRAG):** Progettare un esperimento per mettere a confronto, a parità di modello generativo e di limite massimo di contesto e su domini di entità

allineati, una pipeline RAG documentale classica (basata su PostgreSQL e *pgvector*) con strategie GraphRAG di tipo Text-to-Query (generazione di codice Cypher/SPARQL tramite LLM) e Text-to-Expansion (estrazione deterministica guidata dal modello).

2. Definizione del Dominio e Costruzione dello Scope

La costruzione del Knowledge Graph segue un approccio query-driven, ispirato alla metodologia delle *competency questions* [4], in cui i confini del grafo (*scope*) sono definiti sulla base delle necessità informative delle query (riportate in Sezione 5.1) che hanno portato alla scelta delle proprietà riportate in tabella 2.1.

Tabella 2.1: Proprietà Wikidata utilizzate.

Codice	Nome
P31	instance of
P106	occupation
P18	image
<i>sitelinks</i>	sitelink Wikimedia
P170	creator
P571	inception
P2048, P2049	height, width
P195	collection
P186, P136, P180	material, genre, depicts
P88	commissioned by
P135	movement
P737, P1066	influenced by, student of
P19, P20	place of birth/death
P276, P131	location, admin. territory
P17, P625	country, coordinates
P569, P570	date of birth/death
P245, P1014, P1667, P297	ULAN, AAT, TGN, ISO
<code>rdfs:label</code> , <code>schema:description</code>	testo descrittivo
<i>sitelink Wikipedia</i>	URL della voce

La pipeline definisce uno scope di estrazione delle informazioni riproducibile. Questo processo parte da una selezione di artisti seed di partenza e si espande lungo la rete di relazioni, garantendo la presenza di tutti i nodi e i predicati necessari all'esecuzione del workload di test.

2.1 Selezione del Campione di Artisti di Partenza (Seed)

Abbiamo selezionato un nucleo di 50 artisti "seed" appartenenti al movimento artistico dell'Impressionismo e ai movimenti ad esso collegati, come il Post-Impressionismo, il Simbolismo e i Fauves. Questi artisti e i loro movimenti offrono una rete densa e complessa di scambi, influenze e collaborazioni permettendoci di ottenere delle relazioni non banali o isolate tra i dati.

Per il processo di selezione degli artisti, è stata implementata una procedura composta di tre fasi:

1. **Ricerca dei candidati:** Per ciascun nome di artista viene interrogata l'API di Wikidata (*wbsearchentities*) con lingua di ricerca inglese, limitando il recupero ai primi 5 candidati restituiti in base al punteggio di rilevanza interno del motore di ricerca.
2. **Filtraggio basato sull'occupazione:** Al fine di scartare omonimi ed entità non pertinenti, viene eseguita localmente tramite SPARQL una query di verifica sui candidati per accertare la presenza dell'asserzione diretta *P106* (occupazione) con valore *Q1028181* (pittore).
3. **Validazione e salvataggio:** L'identità risolta corrisponde al primo candidato che soddisfa il vincolo occupazionale. Un'ispezione manuale di controllo delle etichette (label) e delle descrizioni convalida la bontà della risoluzione prima di esportare il file strutturato *artists.json*.

La Tabella 2.2 mostra la suddivisione degli artisti seed in base ai gruppi storici di appartenenza e ai rispettivi QID.

Tabella 2.2: Gruppi del campione seed e identità Wikidata validate (50 pittori in totale).

Gruppo	Artisti e QID
Precursori e mentori (6)	Corot (Q148475), Rousseau (Q310025), Millet (Q148458), Daubigny (Q252357), Boudin (Q212600), Jongkind (Q704585)
Nucleo impressionista (9)	Manet (Q40599), Monet (Q296), Renoir (Q39931), Degas (Q46373), Pissarro (Q134741), Sisley (Q175130), Bazille (Q207298), Caillebotte (Q295144), Guillaumin (Q436844)
Donne dell'Impressionismo (4)	Morisot (Q105320), Cassatt (Q173223), Gonzalès (Q464300), Bracquemond (Q273552)
Cerchia internazionale (4)	Whistler (Q203643), Sargent (Q155626), Sorolla (Q351746), Zorn (Q206820)
Neo-impressionismo (7)	Seurat (Q34013), Signac (Q151573), Luce (Q545293), Angrand (Q1063574), Cross (Q555224), van Rysselberghe (Q671883), Dubois-Pillet (Q2831132)
Post-impressionismo (7)	Cézanne (Q35548), van Gogh (Q5582), Gauguin (Q37693), Toulouse-Lautrec (Q82445), Redon (Q154349), Valadon (Q156889), Utrillo (Q108301)
Pont-Aven e Nabis (6)	Bernard (Q264193), Sérusier (Q326606), Denis (Q440369), Bonnard (Q26408), Vuillard (Q239394), Vallotton (Q123740)
Simbolismo ed espressionismo (2)	Klimt (Q34661), Munch (Q41406)
Transizione fauve (5)	Matisse (Q5589), Derain (Q156272), de Vlaminck (Q241098), Braque (Q153793), Marquet (Q267858)

2.2 Algoritmo di Espansione Semantica

Una volta estratti i 50 QID degli artisti, sono state implementate delle regole e dei vincoli di espansione per popolare lo schema e le varie entità. L'algoritmo esegue un'estrazione strutturata in tre passaggi chiave:

- **Selezione delle opere d'arte:** Per ciascuno dei 50 artisti seed vengono cercate le entità dichiarate come dipinto ($P31 = Q3305213$) associate all'artista ($P170$), provviste di immagine ($P18$). Al fine di evitare che autori con un maggiore numero di opere dominino quantitativamente il grafo introducendo sbilanciamenti, viene posto un limite massimo di 12 opere per artista seed. Le opere candidate vengono ordinate per numero decrescente di *sitelink* (indicatore riproducibile di rilevanza e documentazione) e, in caso di pari merito, viene adottato come tie-break deterministico l'ordinamento crescente del QID numerico. Questo passaggio ha portato alla selezione di 555 opere d'arte.
- **Relazioni tra artisti:** Le query del workload richiedono l'esplorazione delle relazioni tra gli artisti. L'algoritmo interroga le proprietà $P1066$ (allievo di) e $P737$ (influenzato da) degli artisti seed, filtrandone i target per assicurare che si tratti di esseri umani ($P31 = Q5$). Per evitare un'esplosione ricorsiva e non controllata del grafo, l'espansione si arresta ad un solo livello dai seed. Gli artisti referenziati così identificati vengono integrati con i seed, portando a 120 gli artisti totali censiti. Tali entità referenziate fungono da chiusura topologica delle relazioni e non subiscono l'ulteriore estrazione delle opere descritta al punto 1.
- **Estrazione delle entità correlate (Endpoints):** Una volta selezionate le 555 opere, il sistema ha analizzato i collegamenti verso oggetti esterni tramite proprietà specifiche, come la collezione di appartenenza ($P195$), i materiali ($P186$), il genere ($P136$) o il soggetto raffigurato ($P180$). Dal punto di vista della modellazione, questi oggetti non sono considerati semplici attributi testuali, ma vengono considerati nodi indipendenti. Inserendo questi termini come entità dotate di identificativo, label e descrizione, il grafo diventa navigabile in ogni sua direzione: è possibile, ad esempio, passare da un'opera al suo materiale e da lì risalire a tutte le altre opere che condividono la stessa tecnica, trasformando dati isolati in punti di giunzione di una rete strutturata.

2.3 Risoluzione e Normalizzazione Geografica

Uno dei problemi principali nell'uso dei dati grezzi di Wikidata riguarda la disomogeneità. Spesso, le proprietà relative ai luoghi (come la nascita o la collocazione di un museo) non si riferiscono a una città, ma a entità molto più specifiche come ad esempio un quartiere, rendendo errate le operazioni di raggruppamento e confronto spaziale su scala urbana previste dal workload.

Per uniformare questi dati, abbiamo implementato una pipeline di normalizzazione tramite risalita gerarchica. L'obiettivo è ricondurre ogni punto geografico a un'entità urbana riconosciuta seguendo questa logica:

- Per ciascuno dei luoghi di nascita, morte o conservazione originariamente estratti dal grafo, viene verificata l'appartenenza a classi ammesse, come città ($Q515$) o comune ($Q15284$).
- Se l'entità analizzata non appartiene a nessuna di queste classi, l'algoritmo percorre all'indietro la relazione $P131$ risalendo di livello in livello.
- La risalita si arresta non appena viene incontrata una delle classi ammesse o qualora venga raggiunto il limite prefissato di 5 hop, prevenendo così la formazione di cicli. In presenza di percorsi multipli o ambiguità di ramificazione, viene privilegiato deterministicamente il QID con valore numerico minore.

Questo processo di normalizzazione comprime l'eterogeneità dei luoghi iniziali in un insieme controllato, dai quali viene infine ricavata l'entità paese di appartenenza (*P17*).

La Tabella 2.3 sintetizza le cardinalità delle entità logiche che compongono lo scope così generato.

Tabella 2.3: Entità incluse nello scope valutato.

Gruppo	Regola di inclusione	QID
Artisti seed	Campione di partenza	50
Artisti referenziati	Influenza/maestro dei seed, limitato a un livello	86
Artisti distinti	Unione dei gruppi precedenti (16 referenziati erano già seed)	120
Opere	Dipinti dotati di immagine, massimo 12 per seed	555
Collezioni	Musei o collezioni oggetti di P195 delle opere	170
Luoghi	Insediamenti urbani normalizzati tramite risalita amministrativa	239
Paesi	Nazioni o territori oggetti di P17 delle città	30
Termini	Classi descrittive (movimento, materiale, genere, soggetto, committente)	1 040
Totale QID distinti	Unione globale, deduplicata per QID unico	2 103

3. Acquisizione e Trasformazione in ArtGraph RDF

Il processo di acquisizione, filtraggio e normalizzazione semantica dei dati trasforma l'unione dei dump grezzi di Wikidata nel grafo d'integrazione strutturato.

3.1 Download dei Dump RDF Grezzi

Una volta definito lo scope del progetto, il passo successivo è stato il popolamento del grafo con i dati. Per questa fase, invece di limitarci a scaricare solo poche proprietà specifiche tramite query parziali, abbiamo preferito acquisire le informazioni complete di ogni entità direttamente da Wikidata. Questo approccio ci permette di evitare perdite di informazioni e conservare l'intero contesto originale.

Il processo ha previsto l'interrogazione sistematica dell'endpoint di esportazione ufficiale di Wikidata tramite richieste HTTP GET configurate sul pattern:

```
https://www.wikidata.org/wiki/Special:EntityData/<QID>.ttl?flavor=dump
```

L'utilizzo del parametro *flavor=dump* consente di ottenere la serializzazione Turtle [5] completa dell'entità, secondo il modello dati di Wikidata [6]. Questa modalità permette di acquisire l'intero modello degli statement di Wikidata, includendo:

- I nodi intermedi (*p:*) e i relativi valori di statement (*ps:*);
- I qualificatori (*pq:*), necessari per recuperare metadati contestuali (come intervalli temporali);
- Le forme normalizzate dei valori quantitativi (*psn:*);
- I riferimenti di provenienza (*prov:wasDerivedFrom*), utili per tracciare l'origine del dato.

Il processo di download ha riguardato i 2103 QID definiti nello scope. Per garantire la tracciabilità delle operazioni e facilitare il monitoraggio di eventuali errori di rete o timeout, ogni transazione è stata registrata in un log in formato JSONL. I volumi operativi risultanti dalla procedura sono i seguenti:

- **Spazio fisico su disco:** 147.804.825 byte totali, distribuiti su file fisici individuali denominati secondo il rispettivo identificatore (es. *Q273552.ttl*);
- **Volume complessivo dei file individuali:** la somma delle righe dei singoli dump ammonta a 3.198.993 righe di serializzazione. Il dato è un indicatore di volume e non un conteggio di triple, infatti, nella sintassi Turtle una riga non corrisponde necessariamente a una tripla.

- **Unione locale e deduplicazione:** l'unione dei file all'interno di un archivio di memorizzazione locale (implementato mediante il motore in-memory pyoxigraph) ha rimosso le triple duplicate e i metadati di esportazione ridondanti condivisi, restituendo un grafo integrato di 2.830.404 triple uniche;
- **Eterogeneità semantica:** l'unione dei dump ha evidenziato un'elevata complessità strutturale, con 11.051 predicati distinti e campi testuali in oltre 570 lingue.

La struttura misurata sui dump grezzi è schematizzata nella Figura 3.1, evidenziando le connessioni tra entità, statement, nodi valore e metadati di pubblicazione.

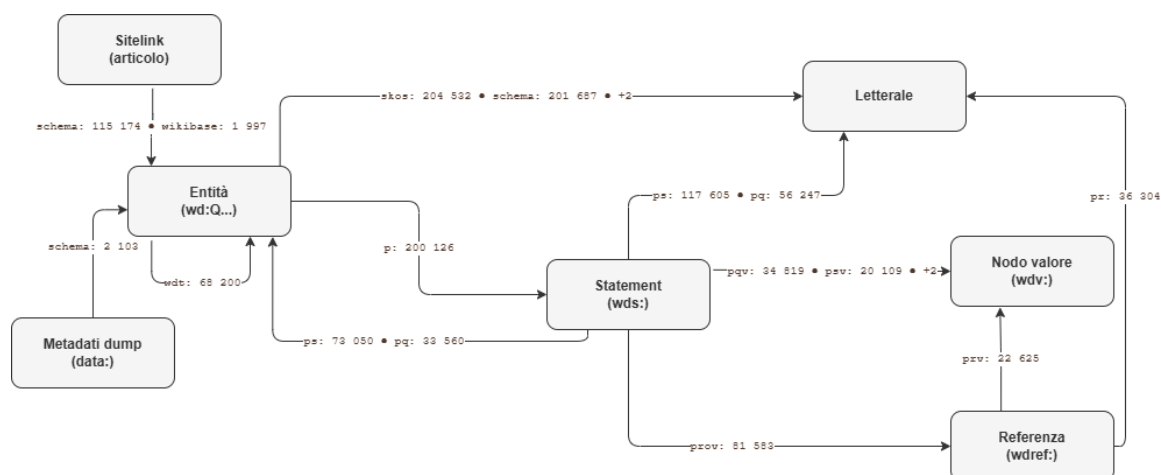


Figura 3.1: Architettura strutturale e flussi di relazione misurati all'interno dei dump RDF completi di Wikidata.

3.2 Trasformazione e Riduzione tramite SPARQL CONSTRUCT

Il volume di 2,8 milioni di triple dei dump grezzi richiede una fase di filtraggio e riduzione per rendere il grafo computazionalmente sostenibile rispetto al workload di test. Sotto il profilo metodologico, la trasformazione è stata eseguita esclusivamente tramite query SPARQL *CONSTRUCT* locali.

L'uso del *CONSTRUCT* garantisce la tracciabilità del dato con due vantaggi principali:

- **Integrità del modello:** A differenza delle *SELECT*, che restituiscono tuple tabellari forzando la ricostruzione manuale di nodi e metadati nel codice, la *CONSTRUCT* mantiene la natura a grafo del dato. Questo permette di operare direttamente sulla semantica, preservando datatype e language tag in modo nativo.
- **Tracciabilità (Mapping):** Ogni tripla generata in *artgraph.ttl* deriva da una regola di estrazione definita, rendendo trasparente il legame tra la sorgente grezza e il database LPG finale.

Il processo è strutturato in otto query (C1-C8), ciascuna dedicata a un sotto-insieme del workload:

- **C1-C2 (Artisti):** Estraggono i dati biografici, gli identificativi Getty ULAN [7] e le relazioni di apprendistato, inclusi i relativi vincoli temporali (*pq:P580*, *pq:P582*).
- **C3-C4 (Opere e Collezioni):** Mappano metadati delle opere (cronologia, immagini, dimensioni normalizzate) e i dettagli relativi all'inventario e ai periodi di acquisizione nelle collezioni museali.
- **C5-C8 (Entità Geografiche e Tassonomiche):** Arricchiscono i nodi relativi a luoghi, paesi e termini classificatori, integrando codici standard ISO, Getty AAT e TGN.

Il seguente esempio riporta la query **C3**, utilizzata per estrarre le opere d'arte, che dimostra la logica di normalizzazione dimensionale e la risoluzione dei link Wikipedia:

Listing 3.1: Query CONSTRUCT C3 per la trasformazione e normalizzazione operativa delle opere.

```

PREFIX ex:      <https://artgraph.univpm.example/ontology#>
PREFIX wd:      <http://www.wikidata.org/entity/>
PREFIX wdt:      <http://www.wikidata.org/prop/direct/>
PREFIX p:        <http://www.wikidata.org/prop/>
PREFIX psn:      <http://www.wikidata.org/prop/statement/value-normalized/>
PREFIX wikibase: <http://wikiba.se/ontology#>
PREFIX rdfs:     <http://www.w3.org/2000/01/rdf-schema#>
PREFIX schema:   <http://schema.org/>

CONSTRUCT {
  ?work rdfs:label ?label .
  ?work wdt:P571 ?inception .      # Data di creazione
  ?work wdt:P18 ?image .           # URL dell'immagine
  ?work ex:heightM ?hM .           # Altezza normalizzata (in metri)
  ?work ex:widthM ?wM .            # Larghezza normalizzata (in metri)
  ?work ex:wikipediaUrl ?article . # Link Wikipedia integrato
}
WHERE {
  VALUES ?work { wd:Q25251 ... } # QID estratti dallo scope

  OPTIONAL { ?work rdfs:label ?label . FILTER(LANG(?label) = "en") }
  OPTIONAL { ?work wdt:P571 ?inception }
  OPTIONAL { ?work wdt:P18 ?image }

  # Estrazione valori normalizzati (psn:) per dimensioni fisiche
  OPTIONAL { ?work p:P2048/psn:P2048/wikibase:quantityAmount ?hM }
  OPTIONAL { ?work p:P2049/psn:P2049/wikibase:quantityAmount ?wM }

  # Risoluzione link alla voce Wikipedia inglese
  OPTIONAL {
    ?art schema:about ?work ;
    schema:isPartOf <https://en.wikipedia.org/> .
    BIND(?art AS ?article)
  }
}

```

Per disaccoppiare il sistema dalla struttura interna di Wikidata, è stato definito un vocabolario locale (*ex:*) documentato nello schema *schema.ttl*. Questo layer di astrazione introduce classi esplicite (es. *ex:Artist*, *ex:Artwork*) e proprietà normalizzate, semplificando le fasi di interrogazione e integrazione geografica.

Il risultato di queste operazioni è il grafo RDF *artgraph.ttl*. La Tabella 3.1 ne riporta una scomposizione: ogni tripla appartiene a una sola categoria e tutti i predicati presenti sono mostrati esplicitamente.

Tabella 3.1: Composizione sintetica ed esaustiva di `artgraph.ttl`.

Categoria	Classe/Predicato	Tipo oggetto	N. triple	% sul totale
Assegnazioni <i>rdf:type</i>	<i>ex:Subject</i>	IRI (classe)	939	4,95%
	<i>ex:Artwork</i>	IRI (classe)	555	2,93%
	<i>ex:Place</i>	IRI (classe)	239	1,26%
	<i>ex:Collection</i>	IRI (classe)	170	0,90%
	<i>ex:Artist</i>	IRI (classe)	120	0,63%
	<i>ex:Movement</i>	IRI (classe)	56	0,30%
	<i>ex:Country</i>	IRI (classe)	30	0,16%
	<i>ex:Genre</i>	IRI (classe)	30	0,16%
	<i>ex:Material</i>	IRI (classe)	19	0,10%
	<i>ex:Patron</i>	IRI (classe)	7	0,04%
Subtotale assegnazioni <i>rdf:type</i>			2 165	11,42%
Proprietà con oggetto IRI	<i>wdt:P180</i>	IRI	2 386	12,59%
	<i>ex:forSource</i>	IRI	1 332	7,03%
	<i>ex:forTarget</i>	(statement) IRI	1 332	7,03%
	<i>wdt:P186</i>	(statement) IRI	1 085	5,72%
	<i>wdt:P195</i>	IRI	576	3,04%
	<i>wdt:P170</i>	IRI	555	2,93%
	<i>wdt:P136</i>	IRI	517	2,73%
	<i>wdt:P135</i>	IRI	335	1,77%
	<i>wdt:P17</i>	IRI	239	1,26%
	<i>ex:locatedInCity</i>	IRI	180	0,95%
	<i>ex:diedInCity</i>	IRI	124	0,65%
	<i>ex:bornInCity</i>	IRI	121	0,64%
	<i>wdt:P1066</i>	IRI	91	0,48%
	<i>wdt:P737</i>	IRI	78	0,41%
	<i>wdt:P88</i>	IRI	7	0,04%
Subtotale proprietà con oggetto IRI			8 958	47,26%
Proprietà con oggetto letterale	<i>ex:inventoryNumber</i>	Letterale (statement)	549	2,90%
	<i>wdt:P571</i>	Letterale	545	2,87%
	<i>ex:heightM</i>	Letterale	531	2,80%
	<i>ex:widthM</i>	Letterale	527	2,78%
	<i>wdt:P1014</i>	Letterale	492	2,60%

Continua nella pagina successiva

Tabella 3.1 (continua dalla pagina precedente)

Categoria	Classe/Predicato	Tipo oggetto	N. triple	% sul totale
	wdt:P625	Letterale	238	1,26%
	ex:since	Letterale	215	1,13%
		(statement)		
	wdt:P1667	Letterale	145	0,76%
	wdt:P569	Letterale	120	0,63%
	wdt:P570	Letterale	120	0,63%
	wdt:P245	Letterale	117	0,62%
	wdt:P297	Letterale	29	0,15%
	ex:until	Letterale	29	0,15%
		(statement)		
	Subtotale proprietà con oggetto letterale		3 657	19,29%
Metadati	rdfs:label	Letterale	2 103	11,09%
	schema:description	Letterale	1 112	5,87%
	wdt:P18	IRI	555	2,93%
	ex:wikipediaUrl	IRI	408	2,15%
	Subtotale metadati		4 178	22,04%
Totale generale			18 958	100,00%

3.3 Integrazione Semantica e Gestione delle Lingue

Il grafo *artgraph.ttl* viene finalizzato applicando regole di normalizzazione semantica per standardizzare i tipi di dato e i valori scalari richiesti dalle query di benchmark.

Normalizzazione dimensionale e geografica. Le dimensioni fisiche sono normalizzate in proprietà monovalore (*ex:heightM*, *ex:widthM*). Le relazioni geografiche (*P19*, *P20*, *P276*, *P131*) sono invece mappate su bridge urbani standardizzati (*ex:bornInCity*, *ex:diedInCity*, *ex:locatedInCity*). Questa astrazione assicura l'uniformità delle aggregazioni spaziali nei tre modelli, escludendo entità a granularità incoerente come singoli edifici o frazioni.

Risoluzione linguistica e selezione deterministica. Poiché nel modello LPG operativo si è scelto di associare a ciascun nodo una proprietà testuale univoca (*name* o *title*), così da rendere prevedibili scansioni e indici, è stata implementata una politica di selezione deterministica dei letterali a priorità decrescente:

1. **Inglese (*en*):** lingua primaria per la standardizzazione del corpus.
2. **Language-neutral (*mul*):** fallback su identificativi internazionali.
3. **Italiano (*it*):** ulteriore livello di localizzazione.
4. **Fallback lessicografico:** in assenza delle lingue precedenti, l'algoritmo seleziona il valore minimo tra tutti i letterali disponibili (ordinamento lessicografico), garantendo l'identificabilità di ogni nodo ed evitando entità anonime.

La medesima logica è applicata alle descrizioni (*schema:description*), mentre i letterali esclusi vengono rimossi per ottimizzare il peso del grafo operativo.

Audit finale dell’artefatto. L’artefatto prodotto è stato verificato ri-parsificando *artgraph.ttl* con un parser RDF conforme e contando le triple sotto semantica insiemistica. Il grafo finale contiene 18.958 triple, distribuite su 3.435 soggetti e 33 predicati distinti, per 1.155.774 byte di serializzazione Turtle.

I 3.435 soggetti si scompongono in 2.103 entità di dominio dotate di QID e 1.332 nodi statement, introdotti per rappresentare in RDF i qualificatori delle relazioni di collezione, apprendistato, influenza e committenza (predicati *ex:forSource* ed *ex:forTarget*, 1.332 occorrenze ciascuno in Tabella 3.1). Analogamente, i 33 predicati comprendono i 28 predicati di dominio più quelli dedicati alla reificazione e ai qualificatori temporali. Il conteggio coincide predicato per predicato con la scomposizione della Tabella 3.1, di cui costituisce la verifica indipendente.

Questi 1.332 nodi statement vanno distinti dalle 752 relazioni che la variante V2 reificherà nel modello a property graph (Sezione 4.3): i due conteggi misurano oggetti differenti. I nodi statement RDF corrispondono agli statement della sorgente e ne conservano la molteplicità, cosicché una medesima coppia soggetto-oggetto può generarne più di uno quando la sorgente documenta, ad esempio, numeri d’inventario o periodi di acquisizione distinti; i 1.332 nodi si riducono infatti a 789 coppie distinte, e 767 di essi recano effettivamente un qualificatore. Le 752 relazioni di V2 sono invece le relazioni logiche deduplicate delle quattro famiglie interessate, e coincidono esattamente con le asserzioni diritte presenti nel grafo: 576 per *P195*, 91 per *P1066*, 78 per *P737* e 7 per *P88*.

4. Modellazione e Traduzione in Property Graph (Neo4j)

La transizione dal modello a triple RDF al paradigma a grafi con proprietà (Labeled Property Graph, LPG) richiede regole sistematiche per convertire la semantica formale di Wikidata in strutture ottimizzate per l'interrogazione tramite il linguaggio Cypher. Si descrive di seguito il mapping formale e le due varianti topologiche V1 e V2, progettate per analizzare l'impatto della reificazione delle relazioni.

4.1 Mapping formale RDF-to-LPG

La traduzione sistematica del sotto-grafo di integrazione ArtGraph nel modello operativo LPG implementato in Neo4j è definita da un insieme di corrispondenze formali. Un grafo RDF [8] composto da triple del tipo (s,p,o) viene mappato in un grafo orientato LPG, definito da nodi dotati di etichette e proprietà, e da archi diretti anch'essi provvisti di etichette e proprietà.

Le regole di mapping comuni a entrambe le versioni di schema sono le seguenti:

1. **Identità dei Nodi e QID:** Ogni risorsa del dominio identificata univocamente da un QID Wikidata viene tradotta in un singolo nodo LPG. L'identificatore originario viene memorizzato come proprietà primaria *qid* del nodo, garantendo che a ogni risorsa corrisponda un'unica entità nel database.
2. **Conversione dei Tipi in Etichette (Multi-Labeling) ed Etichette di Dominio:** Le asserzioni *rdf:type* aventi come oggetto una classe locale vengono mappate direttamente nelle etichette del nodo Neo4j. Qualora una risorsa ricopra più ruoli semantici all'interno del dominio (ad esempio, un artista che è anche un soggetto raffigurato nel dipinto di un altro autore), il nodo LPG riceve l'unione di tali etichette (Multi-Labeling), preservando l'identità globale dell'entità senza duplicarla. Le etichette di dominio ammesse nel grafo sono 10: *Artist*, *Artwork*, *Movement*, *Collection*, *Place*, *Country*, *Material*, *Genre*, *Subject* e *Patron*.
3. **Mappatura delle Proprietà Scalari e Normalizzazione Nomi:** I valori letterali associati a proprietà descrittive e identificativi esterni sono convertiti in proprietà scalari del nodo LPG corrispondente. Nello specifico, date e anni vengono convertiti nei formati operativi, mentre le coordinate geografiche (*wdt:P625*) e gli identificativi (codici ULAN, AAT, TGN, ISO) sono salvati sotto forma di stringhe. Ogni etichetta descrittiva RDF (*rdfs:label*) viene mappata nella proprietà *name* del nodo, anche per le opere d'arte; la voce *title* non viene memorizzata come proprietà del nodo, ma resta riservata unicamente come alias per le colonne restituite dalle query Cypher.
4. **Trattamento degli IRI Esterni:** Gli IRI esterni che rappresentano attributi e non entità autonome del dominio (quali l'URL dell'immagine in *wdt:P18* o il link alla voce enciclopedica in *ex:wikipediaUrl*) vengono convertiti direttamente in proprietà di tipo stringa salvate all'interno del nodo soggetto.

5. **Mappatura delle Relazioni (Archivi Orientati) e Deduplicazione:** I predicati RDF orientati che collegano due risorse distinte provviste di QID vengono convertiti in relazioni dirette e tipizzate in Neo4j (es. *CREATED*, *IN_COLLECTION*, *USES_MATERIAL*). Nel processo di conversione, eventuali coppie di archi identiche e duplicate vengono eliminate; al contrario, la presenza di valori multipli distinti associati allo stesso predicato genera più relazioni separate nel database.

Le corrispondenze tra le etichette di dominio LPG e le proprietà operative aggiuntive ad esse associate (escluse le chiavi comuni *qid* e *wikipedia_url*) sono riassunte nella Tabella 4.1.

Etichetta	Proprietà operative aggiuntive
Artist	name, description, birth_date, death_date, ulan_id
Artwork	name, description, inception_year, height_m, width_m, image_url
Collection	name
Place	name, description, coordinates, iso_code, tgn_id
Country	name, description, coordinates, iso_code, tgn_id
Movement	name, description, aat_id
Material	name, description, aat_id
Genre	name, description, aat_id
Subject	name, description, aat_id
Patron	name, description, birth_date, death_date, ulan_id

Tabella 4.1: Attributi descrittivi e proprietà operative dei nodi del dominio.

4.2 Modello V1: Baseline Entity-Centric

Il modello V1 costituisce la baseline dell'architettura dati, progettata seguendo un approccio entity-centric. In questa versione, ogni relazione concettuale del dominio operativo viene materializzata direttamente come arco orientato (direct edge) tra nodi entità. Lo schema evita la reificazione tramite nodi intermedi, mappando 1:1 le 6294 relazioni logiche del dominio in altrettanti archi diretti. Tale scelta architetturale riduce la complessità sintattica delle query Cypher e ottimizza le operazioni di one-hop, in quanto la risoluzione dei vicini richiede la scansione di un solo arco. I qualificatori e i metadati contestuali sono incapsulati direttamente come proprietà degli archi (edge properties). La relazione tra un'opera d'arte e la relativa collezione è formalizzata come segue:

(Artwork) -[:IN_COLLECTION {since: '1894', inventory_number: 'RF 2771'}]-> (Collection)

La Figura 4.1 illustra lo schema logico del modello V1.

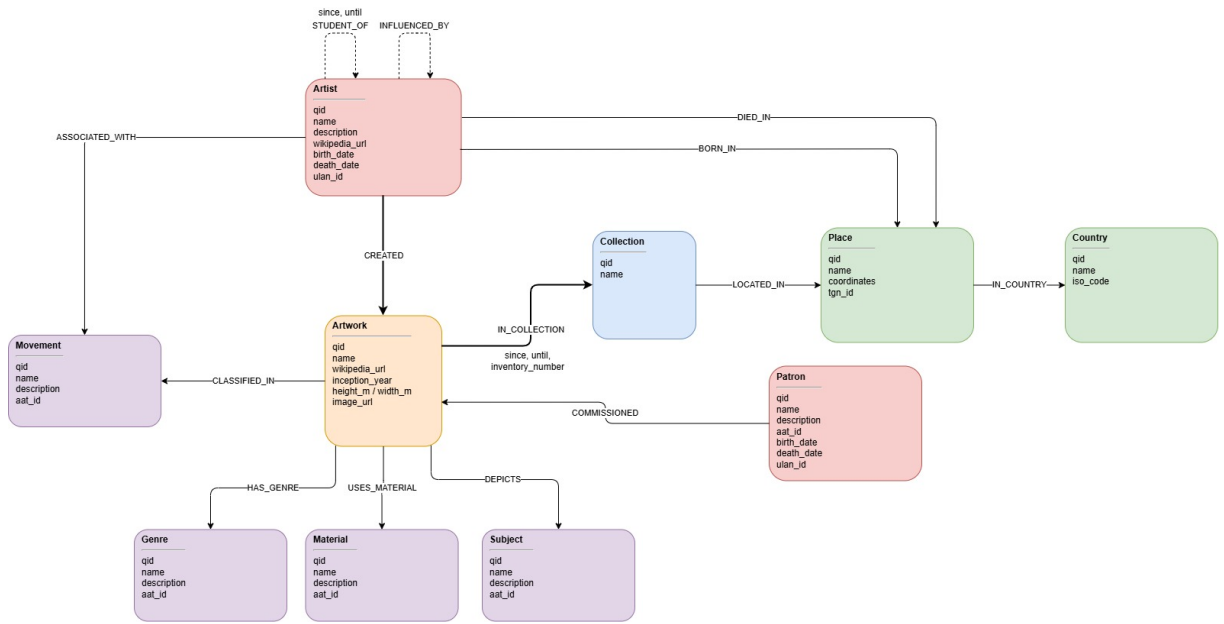


Figura 4.1: Schema Neo4j V1.

4.3 Modello V2: Reificazione delle Relazioni

La seconda variante adotta una strategia di reificazione selettiva applicata a quattro specifiche famiglie di relazioni: collezione (*P195*), maestro-allievo (*P1066*), influenza (*P737*) e committenza (*P88*). Ciascuna delle 752 istanze logiche afferenti a tali categorie viene de-strutturata: al posto di un singolo arco diretto, la relazione è modellata interponendo un nodo intermedio (denominato nodo statement) collegato alle entità di dominio tramite due archi orientati. Le quattro classi di nodi statement introdotte nello schema sono:

- *CollectionStatement*: reifica l'appartenenza di un'opera a una collezione;
- *ApprenticeshipStatement*: reifica il rapporto tra un allievo e un maestro;
- *InfluenceStatement*: reifica l'influenza artistica intercorrente tra due autori;
- *CommissionStatement*: reifica il vincolo di committenza tra mecenate e opera.

A livello strutturale, i nodi statement sono privi di identificativi QID o proprietà intrinseche. Tuttavia, nei casi in cui i dati sorgente presentino qualificatori temporali o gestionali (nello specifico per *IN_COLLECTION* e *STUDENT_OF*), il posizionamento delle proprietà scalari segue una distinzione dettata dal significato semantico del dato:

- **Reificazione su Nodo (*IN_COLLECTION*):** I metadati *since*, *until* e *inventory_number* vengono allocati come proprietà interne del nodo *CollectionStatement*. Tale scelta modella l'affermazione come un'entità autonoma, assimilabile a un record di acquisizione o scheda museale.
- **Reificazione su Arco (*STUDENT_OF*):** I qualificatori temporali *since* e *until* sono posizionati sull'arco uscente *TO_MASTER*, che connette il nodo *ApprenticeshipStatement* al nodo del maestro. Questa scelta modella la componente temporale come attributo proprio della relazione.

Questo design consente di misurare l’overhead prestazionale introdotto dai salti aggiuntivi (multi-hop traversal) durante l’esecuzione delle query, mantenendo invariata l’equivalenza informativa rispetto alla baseline V1. Lo schema del modello V2 è illustrato in Figura 4.2.

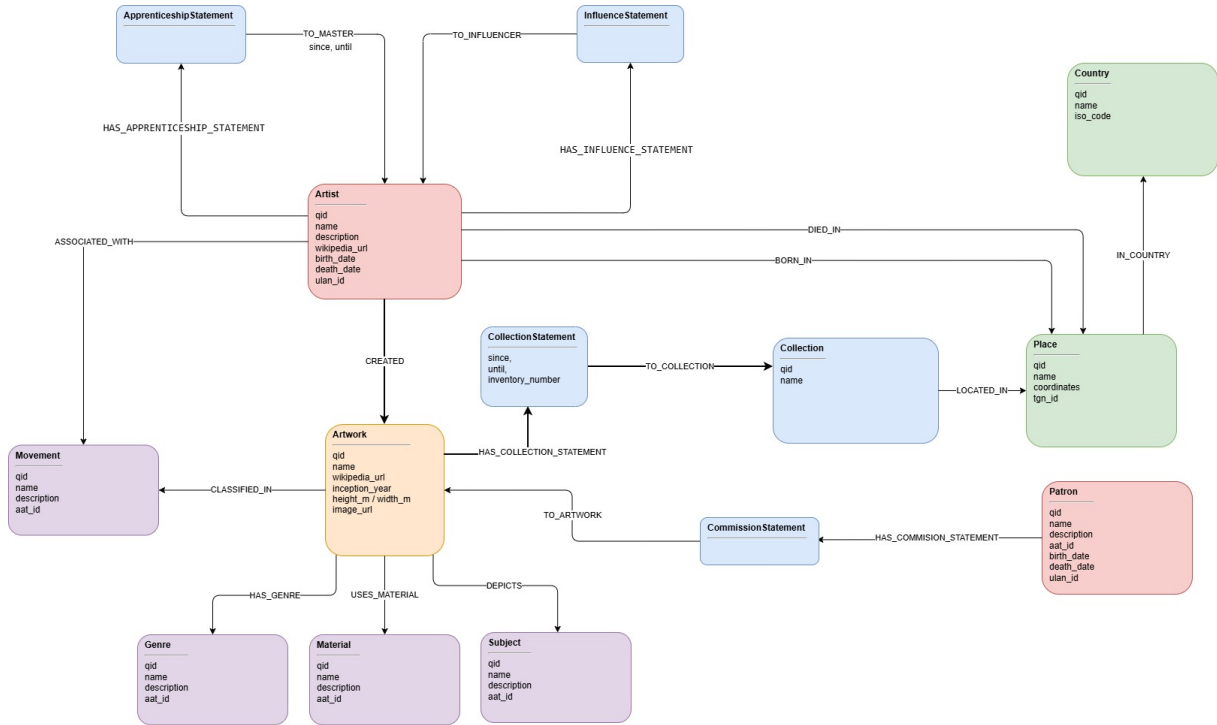


Figura 4.2: Schema Neo4j V2.

4.4 Pipeline di Ingestione e Validazione

Il caricamento delle due versioni dello schema all’interno di Neo4j si articola in una pipeline di ETL (Extract, Transform, Load) riproducibile. Per garantire l’isolamento dei benchmark e l’accuratezza delle metriche di allocazione fisica della memoria, l’ingestione di ciascuna versione è preceduta da una procedura di wipe del database, la quale rimuove integralmente nodi, archi, vincoli e indici preesistenti. La strategia di scrittura adotta un pattern di batch processing basato sull’operatore Cypher *UNWIND*. Vengono trasmessi al DBMS elenchi parametrizzati di dizionari Python, eseguendo le transazioni in blocchi atomici. Tale approccio riduce l’overhead di rete derivante dai round-trip client-server e previene fenomeni di saturazione dell’heap della JVM (Java Virtual Machine). La pipeline di caricamento si articola nelle seguenti fasi sequenziali:

1. **Inizializzazione dello Schema:** Viene applicato un vincolo di unicità (Uniqueness Constraint) sull’attributo *qid* della classe base *Entity* e vengono generati gli indici di ricerca sulla proprietà *name* per tutte le etichette di dominio. L’esecuzione dei job successivi viene bloccata fino al completamento dell’indicizzazione in background.
2. **Ingestione dei Nodi Entità:** I 2103 nodi strutturati, corredati dai rispettivi attributi scalari, vengono scritti nel database mediante clausole *MERGE* indicizzate sulla chiave primaria *qid*.
3. **Materializzazione delle Relazioni e degli Statement:** In base allo schema target V1 o V2, lo script processa l’insieme delle relazioni logiche generandone la topologia corrispondente: archi diretti single-hop per la baseline V1, ovvero nodi statement intermedi con biforcazione two-hop per la variante V2.

A valle dell'ingestione, si esegue un audit strutturale incrociato per verificare l'integrità dei dati rispetto alla sorgente RDF originale. La procedura accerta l'assenza di data loss, la corrispondenza dei QID e l'equivalenza semantica delle 6294 relazioni logiche ricostruite in entrambe le versioni. Le metriche di storage e la sintesi quantitativa dell'ingestione sono riportate nella Tabella 4.2.

Versione	Nodi	Relazioni	Proprietà	Stima minima record store (KiB)
V1 (Tutte dirette)	2,103	6,294	9,872	635
V2 (Reificate)	2,855	7,046	9,872	671

Tabella 4.2: Cardinalità misurate e stima dell'overhead strutturale di storage per le versioni V1 e V2.

L'ultima colonna non è una misura dell'occupazione reale su disco, bensì una stima analitica derivata dalle cardinalità tramite le dimensioni fisse dei record del formato standard di Neo4j¹. Ad esempio, assumendo 15 byte per nodo, 34 per relazione e 41 per proprietà, la stima è calcolata secondo l'Equazione 4.1.

$$\text{stima} = 15 n_{\text{nodi}} + 34 n_{\text{rel}} + 41 n_{\text{prop}} \quad (4.1)$$

da cui si ottengono 650.293 byte (635 KiB) per V1 e 687.141 byte (671 KiB) per V2. Si tratta di un limite inferiore: i valori lunghi, come stringhe e array, eccedono il record di proprietà e sono delegati agli *store* dinamici, mentre indici e *transaction log* sono esclusi dal computo. La dimensione effettiva dei file di store non è stata misurata per una ragione metodologica: i file di Neo4j non si contraggono a seguito delle cancellazioni, poiché i record eliminati vengono soltanto resi riutilizzabili; in un ciclo di svuotamento e ricaricamento ogni versione erediterebbe quindi il massimo storico raggiunto dalla precedente, restituendo un confronto privo di significato. La grandezza confrontata è pertanto l'overhead strutturale imputabile alla topologia, non l'occupazione fisica né la memoria residente del processo.

¹<https://neo4j.com/developer/kb/understanding-data-on-disk/>

5. Benchmark Prestazionale del Workload

Il benchmark valuta le prestazioni di Neo4j V1, Neo4j V2 e Oxigraph RDF su un workload di 35 query. Il protocollo misura latenze di risposta e accessi logici (*database hits*) per identificare i trade-off computazionali tra i diversi modelli e l'impatto delle dinamiche di allocazione fisica sulla riproducibilità delle metriche.

5.1 Le 35 Query

La validazione dello schema e la verifica dell'espressività informativa del Knowledge Graph si articolano su un set di 35 query, formalizzate ex-ante rispetto alla fase di ingestione dei dati, definendo lo scope dell'architettura. Le 35 query sono riportate di seguito:

- Q1.** Recuperare le informazioni anagrafiche di un artista.
- Q2.** Ottenere le informazioni di una certa opera d'arte.
- Q3.** Elencare i materiali di cui è composta una determinata opera d'arte.
- Q4.** Trovare per una determinata collezione in quale città si trova e le sue coordinate geografiche.
- Q5.** Elencare i dipinti di una determinata corrente artistica, con il rispettivo anno di creazione.
- Q6.** Trovare gli artisti influenzati da un certo maestro.
- Q7.** Elencare le opere di un artista e i paesi in cui sono conservate.
- Q8.** Trovare i maestri di un artista e i movimenti artistici a cui tali maestri appartengono.
- Q9.** Trovare le opere di un artista conservate nella stessa città in cui l'artista è nato, con la relativa collezione.
- Q10.** Trovare i movimenti artistici delle opere che raffigurano un determinato soggetto.
- Q11.** Elencare le opere di un certo genere pittorico create da artisti deceduti in una specifica città.
- Q12.** Identificare i committenti che hanno commissionato opere a uno specifico artista, con i titoli delle opere.
- Q13.** Trovare altri dipinti che raffigurano lo stesso soggetto di un'opera data, con i rispettivi autori.
- Q14.** Trovare le collezioni che conservano opere realizzate con un determinato materiale e appartenenti a un certo movimento artistico.
- Q15.** Identificare gli artisti nati e morti nello stesso paese, con i movimenti artistici a cui sono associati.
- Q16.** Per le opere di un artista, trovare la collezione che le conserva con il periodo di acquisizione e il numero di inventario, se documentati.

- Q17. Elencare i materiali e i generi pittorici impiegati nelle opere che raffigurano un determinato soggetto.
- Q18. Trovare i movimenti artistici degli artisti influenzati da un determinato pittore.
- Q19. Trovare le collezioni di un paese che conservano opere realizzate dagli allievi di un certo maestro.
- Q20. Trovare le opere di dimensioni che superano una certa altezza o larghezza, con le rispettive misure, l'autore e la collezione che le conserva.
- Q21. Ottenere, per ciascun movimento artistico, il numero di opere e l'area media, massima e minima dei dipinti.
- Q22. Trovare i dieci soggetti più frequentemente raffigurati nelle opere di un determinato movimento.
- Q23. Ricostruire la catena di influenza, con tutti i passaggi intermedi, che collega un artista a un maestro di riferimento anche indirettamente, entro quattro passaggi.
- Q24. Ordinare i committenti in base al numero di opere commissionate, mostrando i primi cinque con i rispettivi artisti.
- Q25. Trovare coppie di artisti nati nella stessa città e associati allo stesso movimento artistico.
- Q26. Identificare i materiali più utilizzati dagli artisti nati in un determinato paese, ordinati per frequenza d'uso.
- Q27. Trovare i dipinti la cui larghezza superi significativamente l'altezza, con le rispettive misure, l'autore e la collezione.
- Q28. Trovare gli allievi di un maestro.
- Q29. Trovare gli allievi associati ad almeno un movimento artistico diverso da quelli del proprio maestro.
- Q30. Contare quante opere di ciascun genere pittorico sono state create in ogni decennio.
- Q31. Identificare coppie di artisti che sono stati influenzati da almeno due stessi predecessori, indicandone i nomi.
- Q32. Individuare per una coppia di artisti quanti collegamenti distinti si trovano entro quattro passaggi e qual è la distanza minima che li separa.
- Q33. Identificare gli artisti con almeno un'opera conservata in un paese diverso sia da quello di nascita sia da quello di morte, indicando in quali paesi si trovano tali opere.
- Q34. Trovare i committenti che hanno commissionato opere ad artisti di almeno due movimenti artistici diversi, elencando i movimenti coinvolti.
- Q35. Classificare i paesi in base al numero di movimenti artistici distinti rappresentati nelle collezioni che custodiscono.

Per valutare le prestazioni delle diverse topologie di schema al crescere della complessità relazionale del workload, il benchmark è stato categorizzato in cinque classi. Si tratta di una caratterizzazione rispetto alla complessità dell'interrogazione e non della scalabilità dei sistemi rispetto alla dimensione del grafo: il dataset resta invariato in tutte le misurazioni (2.103 entità, 6.294 relazioni in V1 e 7.046 in V2), e una verifica di scalabilità richiederebbe repliche del dataset a fattori crescenti con le relative curve di latenza e occupazione.

- **Descriptive (base):** Query focalizzate sulla ricerca puntuale (lookup) di entità e sulla restituzione dei loro attributi scalari diretti (es. Q01, Q02). Verificano il corretto grounding e la consistenza delle proprietà anagrafiche, dimensionali e biografiche.

- **One-Hop:** Operazioni tra archi adiacenti (es. Q03–Q06, Q28), volte a risolvere associazioni immediate (ad esempio la relazione opera-materiale o artista-movimento).
- **Multi-Hop:** Navigazioni nel grafo a profondità fissa pari o superiore a due salti (≥ 2 hops, es. Q07–Q20, Q29, Q33). Richiedono la concatenazione logica di cammini tra entità eterogenee (es. la risoluzione del percorso da un’opera alla città natale del suo autore).
- **Analytic:** Query ad elevata intensità computazionale caratterizzate da elaborazioni di tipo OLAP, quali aggregazioni statistiche, raggruppamenti per finestre temporali, ordinamenti e classificazioni vincolate (es. Q21, Q22, Q24–Q27, Q30, Q31, Q34, Q35).
- **Path:** Algoritmi di path-finding per l’individuazione di cammini a lunghezza variabile (variable-length paths) volte a rilevare linee di influenza o connessioni indirette, impostando un upper-bound di profondità pari a quattro salti (Q23, Q32).

Il workload opera su un sottoinsieme delimitato di dati; i risultati non sono quindi asserzioni ontologiche universali. La Tabella 5.1 specifica per ogni query i parametri di input adottati nel benchmark, un estratto del payload di output per la variante reificata V2 e la cardinalità totale ($|R|$) del result set.

Tabella 5.1: Esempi eseguibili e output osservati del workload.

Id	Parametri di esempio	Output osservato (estratto)	$ R $
Q01	artista = Claude Monet	Claude Monet; 1840-11-14, Paris; 1926-12-05, Giverny; ULAN 500019484	1
Q02	opera = Olympia	Olympia; 130.5 × 191.0 cm; 1863; upload.wikimedia.org/.../Edouard_Manet...jpg	1
Q03	opera = Olympia	canvas; oil paint	2
Q04	collezione = Musée d’Orsay	Paris; Point(2.35222222222222 48.8566666666667)	1
Q05	movimento = Impressionism	A Bar at the Folies-Bergère (1882); Pont Boieldieu in Rouen, Rainy Weather (1896); ...	51
Q06	influenza = Gustave Courbet	Charles-François Daubigny; Eugène Louis Boudin; Henri-Edmond Cross; ...	7
Q07	artista = Claude Monet	Impression, Sunrise — France; Garden at Sainte-Adresse — United States; ...	11
Q08	artista = Claude Monet	Charles Gleyre — [Orientalist painting]; Johan Jongkind — []; Eugène Louis Boudin — [Impressionism]	3
Q09	artista = Claude Monet	Impression, Sunrise — Musée Marmottan Monet, Paris; Women in the Garden — Musée d’Orsay, Paris; ...	5
Q10	soggetto = woman	hispanism; Impressionism; French Realism; ...	24
Q11	genere = portrait; città = Paris	Georges Seurat — The Models; Charles-François Daubigny — Honoré Daumier; ...	22
Q12	artista commissionato = Henri Matisse	Sergei Shchukin — Music	1
Q13	opera = Olympia	Luncheon on the Grass — Édouard Manet	1

Id	Parametri di esempio	Output osservato (estratto)	 R
Q14	materiale = oil paint; movimento = Impressionism	Courtauld Gallery; Art Gallery of Ontario; Hermitage Museum; ...	27
Q15	nessun filtro	Caspar David Friedrich — Germany — [German Romanticism]; Berthe Morisot — France — [Impressionism]; ...	100
Q16	artista = Claude Monet	Women in the Garden — Musée d'Orsay — inv. RF 2773; Madame Monet wearing a kimono — Museum of Fine Arts Boston — dal 1956-03-08, inv. 56.147; ...	11
Q17	soggetto = woman	13 materiali distinti, fra cui oil paint e canvas; 20 generi distinti, fra cui portrait e landscape painting	1
Q18	influenza = Gustave Courbet	Charles-François Daubigny — Barbizon school; Eugène Louis Boudin — Impressionism; ...	8
Q19	maestro = Camille Pissarro; paese = France	Musée d'Orsay — Paul Gauguin	1
Q20	soglia = 200 cm	Orpheus dismembered, 250 × 200 cm — Félix Vallotton — Museum of Art and History Geneva; ...	48
Q21	nessun filtro	Impressionism: 50 opere; area media 10 540, massima 58 610, minima 2 103 cm ² ; ...	32
Q22	movimento = Impressionism	woman: 23; dress: 12; tree: 10; ...	10
Q23	capostipite = Gustave Courbet	[Maurice Utrillo, Gustave Courbet], 1 salto; [Alfred Sisley, Gustave Courbet], 1 salto; ...	30
Q24	nessun filtro	Ambroise Vollard: 1 opera, [André Derain]; Ferdinand Bloch-Bauer: 1, [Gustav Klimt]; ...	5
Q25	nessun filtro	Jean-Baptiste Camille Corot / Charles-François Daubigny — Paris — Barbizon school; ...	47
Q26	paese = France	oil paint: 396 usi; canvas: 356; cardboard: 25; ...	14
Q27	rapporto minimo = 1.5	The Port at Lorient, 73.0 × 43.5 cm — Berthe Morisot — National Gallery of Art; ...	68
Q28	maestro = Camille Pissarro	Paul Gauguin (periodo di apprendistato non documentato nella sorgente)	1
Q29	nessun filtro	Alfred Stevens [French Realism] — maestro Jean-Auguste-Dominique Ingres [Neoclassicism, Romanticism]; ...	41
Q30	nessun filtro	1800 — portrait: 1; 1810 — landscape painting: 1; 1820 — Veduta: 1; ...	126
Q31	nessun filtro	Paul Gauguin / Vincent van Gogh — [Paul Cézanne, Katsushika Hokusai]; ...	8
Q32	Claude Monet; Camille Pissarro	35 cammini; lunghezza logica minima 2	1
Q33	nessun filtro	Édouard Vuillard — [Brazil, Russia, United States]; Albert Marquet — [Russia, United States]; ...	46

Id	Parametri di esempio	Output osservato (estratto)	 R
Q34	nessun filtro	Étienne Moreau-Nélaton — [Symbolism, Les Nabis, Pont-Aven School]; Sergei Shchukin — [Neo-impressionism, Post-impressionism, divisionism, Fauvism, Impressionism]; ...	5
Q35	nessun filtro	United States: 20 movimenti; France: 17; United Kingdom: 9; ...	15

Vengono di seguito riportate le implementazioni delle 35 query. Al fine di garantire l'equità del confronto, le interrogazioni sono state tradotte nei rispettivi linguaggi nativi preservandone la semantica e l'output atteso.

Nel modello V1, le entità e le relazioni sono tradotte in un property graph diretto, in cui le query sono espresse in linguaggio Cypher. Di seguito è riportato il codice per il set di query, parametrizzate per l'esecuzione a runtime:

```
// Q01: Informazioni anagrafiche artista
MATCH (a:Artist {name: $artist})
OPTIONAL MATCH (a)-[:BORN_IN]->(bp:Place)
OPTIONAL MATCH (a)-[:DIED_IN]->(dp:Place)
RETURN a.name AS artist, a.birth_date AS born, bp.name AS birth_place,
        a.death_date AS died, dp.name AS death_place, a.uln_id AS uln

// Q02: Informazioni opera d'arte
MATCH (w:Artwork {name: $artwork})
RETURN w.name AS title, w.height_m AS height_m, w.width_m AS width_m,
        w.inception_year AS year, w.image_url AS image

// Q03: Materiali di un'opera
MATCH (:Artwork {name: $artwork})-[:USES_MATERIAL]->(m:Material)
RETURN m.name AS material

// Q04: Città e coordinate di una collezione
MATCH (:Collection {name: $collection})-[:LOCATED_IN]->(p:Place)
RETURN p.name AS city, p.coordinates AS coordinates

// Q05: Dipinti di una corrente artistica
MATCH (w:Artwork)-[:CLASSIFIED_IN]->(:Movement {name: $movement})
RETURN w.name AS title, w.inception_year AS year

// Q06: Artisti influenzati da un maestro
MATCH (x:Artist)-[:INFLUENCED_BY]->(:Artist {name: $influencer})
RETURN x.name AS artist

// Q07: Opere di un artista e paesi di conservazione
MATCH (:Artist {name: $artist})-[:CREATED]->(w:Artwork)
        -[:IN_COLLECTION]->(:Collection)-[:LOCATED_IN]->(:Place)
        -[:IN_COUNTRY]->(k:Country)
RETURN DISTINCT w.name AS title, k.name AS country

// Q08: Maestri di un artista e loro movimenti
MATCH (:Artist {name: $artist})-[:STUDENT_OF]->(m:Artist)
OPTIONAL MATCH (m)-[:ASSOCIATED_WITH]->(mv:Movement)
RETURN m.name AS master, collect(DISTINCT mv.name) AS movements

// Q09: Opere conservate nella città natale dell'autore
MATCH (a:Artist {name: $artist})-[:BORN_IN]->(p:Place)
        <-[:LOCATED_IN]-(c:Collection)<-[:IN_COLLECTION]-(w:Artwork)
        <-[:CREATED]-(a)
RETURN w.name AS title, c.name AS collection, p.name AS city

// Q10: Movimenti artistici per soggetto
MATCH (:Subject {name: $subject})<-[:DEPICTS]-(a:Artwork)
```

```

-[:CLASSIFIED_IN]->(m:Movement)
RETURN DISTINCT m.name AS movement

// Q11: Opere per genere e città di morte
MATCH (w:Artwork)-[:HAS_GENRE]->(:Genre {name: $genre}),
      (w)<-[:CREATED]-(a:Artist)-[:DIED_IN]->(:Place {name: $city})
RETURN a.name AS artist, w.name AS title

// Q12: Committenti di uno specifico artista
MATCH (p:Patron)-[:COMMISSIONED]->(w:Artwork)
      <-[:CREATED]-(a:Artist {name: $commissioned_artist})
RETURN p.name AS patron, w.name AS title

// Q13: Altre opere con lo stesso soggetto
MATCH (:Artwork {name: $artwork})-[:DEPICTS]->(s:Subject)
      <-[:DEPICTS]-(o:Artwork)<-[:CREATED]-(a:Artist)
WHERE o.name <> $artwork
RETURN DISTINCT o.name AS title, a.name AS artist

// Q14: Collezioni per materiale e movimento
MATCH (c:Collection)<-[:IN_COLLECTION]-(w:Artwork)
      -[:USES_MATERIAL]->(:Material {name: $material}),
      (w)-[:CLASSIFIED_IN]->(:Movement {name: $movement})
RETURN DISTINCT c.name AS museum

// Q15: Artisti nati e morti nello stesso paese
MATCH (a:Artist)-[:BORN_IN]->(:Place)-[:IN_COUNTRY]->(k:Country)
MATCH (a)-[:DIED_IN]->(:Place)-[:IN_COUNTRY]->(k)
RETURN a.name AS artist, k.name AS country,
      [(a)-[:ASSOCIATED_WITH]->(m:Movement) | m.name] AS movements

// Q16: Opere, collezioni e dati di acquisizione
MATCH (:Artist {name: $artist})-[:CREATED]->(w:Artwork)
      -[:IN_COLLECTION]->(c:Collection)
RETURN w.name AS title, c.name AS collection,
      r.since AS since, r.until AS until, r.inventory_number AS inventory_number

// Q17: Materiali e generi per soggetto
MATCH (:Subject {name: $subject})<-[:DEPICTS]-(w:Artwork)
OPTIONAL MATCH (w)-[:USES_MATERIAL]->(m:Material)
OPTIONAL MATCH (w)-[:HAS_GENRE]->(g:Genre)
RETURN collect(DISTINCT m.name) AS materials, collect(DISTINCT g.name) AS genres

// Q18: Movimenti di artisti influenzati da un pittore
MATCH (x:Artist)-[:INFLUENCED_BY]->(:Artist {name: $influencer}),
      (x)-[:ASSOCIATED_WITH]->(m:Movement)
RETURN DISTINCT x.name AS artist, m.name AS movement

// Q19: Collezioni estere con opere di allievi di un maestro
MATCH (s:Artist)-[:STUDENT_OF]->(:Artist {name: $master}),
      (s)-[:CREATED]->(:Artwork)-[:IN_COLLECTION]->(c:Collection)
      -[:LOCATED_IN]->(:Place)-[:IN_COUNTRY]->(:Country {name: $country})
RETURN DISTINCT c.name AS museum, s.name AS student

// Q20: Opere oltre una certa dimensione
MATCH (a:Artist)-[:CREATED]->(w:Artwork)
WHERE w.height_m >= $min_size_m OR w.width_m >= $min_size_m
OPTIONAL MATCH (w)-[:IN_COLLECTION]->(c:Collection)
RETURN w.name AS title, w.height_m AS height_m, w.width_m AS width_m,
      a.name AS artist, c.name AS museum

// Q21: Statistiche sulle aree per movimento (Analitica)
MATCH (w:Artwork)-[:CLASSIFIED_IN]->(m:Movement)
WHERE w.height_m IS NOT NULL AND w.width_m IS NOT NULL
WITH m, w, round(w.height_m * 10000) * round(w.width_m * 10000) / 100000000.0 AS area_m2
RETURN m.name AS movement, count(w) AS works,

```

```

        round(avg(area_m2), 4) AS avg_area_m2,
        round(max(area_m2), 4) AS max_area_m2,
        round(min(area_m2), 4) AS min_area_m2
ORDER BY works DESC, movement

// Q22: Soggetti piu frequenti per movimento
MATCH (:Movement {name: $movement})<-[:CLASSIFIED_IN]-(:Artwork)-[:DEPICTS]->(s:Subject)
RETURN s.name AS subject, count(*) AS frequency
ORDER BY frequency DESC, subject LIMIT 10

// Q23: Catena di influenza (Path analysis)
MATCH path = (a:Artist)-[:INFLUENCED_BY*1..4]->(:Artist {name: $influencer})
RETURN [n IN nodes(path) | n.name] AS influence_line, length(path) AS hops

// Q24: Top 5 committenti
MATCH (p:Patron)-[:COMMISSIONED]->(w:Artwork)<-[:CREATED]->(a:Artist)
RETURN p.name AS patron, count(DISTINCT w) AS works, collect(DISTINCT a.name) AS artists
ORDER BY works DESC, patron LIMIT 5

// Q25: Coppie di artisti per citta e movimento
MATCH (a1:Artist)-[:BORN_IN]->(p:Place)<-[:BORN_IN]->(a2:Artist),
      (a1)-[:ASSOCIATED_WITH]->(m:Movement)<-[:ASSOCIATED_WITH]->(a2)
WHERE a1.qid < a2.qid
RETURN a1.name AS artist1, a2.name AS artist2, p.name AS city, m.name AS movement

// Q26: Materiali per paese d'origine
MATCH (a:Artist)-[:BORN_IN]->(p:Place)-[:IN_COUNTRY]->(c:Country {name: $country}),
      (a)-[:CREATED]->(w:Artwork)-[:USES_MATERIAL]->(m:Material)
RETURN m.name AS material, count(*) AS uses
ORDER BY uses DESC, material

// Q27: Dipinti sproporzionati in larghezza
MATCH (a:Artist)-[:CREATED]->(w:Artwork)
WHERE round(w.width_m, 4) >= round(w.height_m * $ratio, 4)
OPTIONAL MATCH (w)-[:IN_COLLECTION]->(c:Collection)
RETURN w.name AS title, w.width_m AS width_m, w.height_m AS height_m,
      a.name AS artist, c.name AS museum

// Q28: Allievi e temporalita dell'apprendistato
MATCH (s:Artist)-[:STUDENT_OF]->(r:Artist {name: $master})
RETURN s.name AS student, r.since AS since, r.until AS until

// Q29: Allievi con movimenti diversi dal maestro
MATCH (s:Artist)-[:STUDENT_OF]->(m:Artist)
WITH s, m,
      [(s)-[:ASSOCIATED_WITH]->(x:Movement) | x.name] AS sm,
      [(m)-[:ASSOCIATED_WITH]->(y:Movement) | y.name] AS mm
WHERE size(mm) > 0 AND any(x IN sm WHERE NOT x IN mm)
RETURN s.name AS student, sm AS student_movements,
      m.name AS master, mm AS master_movements

// Q30: Opere per genere per decennio
MATCH (w:Artwork)-[:HAS_GENRE]->(g:Genre)
WHERE w.inception_year IS NOT NULL
RETURN (w.inception_year / 10) * 10 AS decade, g.name AS genre, count(*) AS works
ORDER BY decade, works DESC, genre

// Q31: Coppie influenzate dagli stessi predecessori
MATCH (a1:Artist)-[:INFLUENCED_BY]->(x:Artist)<-[:INFLUENCED_BY]->(a2:Artist)
WHERE a1.qid < a2.qid
WITH a1, a2, collect(DISTINCT x.name) AS shared
WHERE size(shared) >= 2
RETURN a1.name AS artist1, a2.name AS artist2, shared

// Q32: Distanza logica minima (Path Analysis)
MATCH path = (:Artist {name: $artist})

```

```

-[:CREATED|INFLUENCED_BY|STUDENT_OF*1..4]-
(:Artist {name: $artist2})
RETURN count(path) AS paths, min(length(path)) AS shortest

// Q33: Artisti con opere in paesi stranieri (ne di nascita ne di morte)
MATCH (a:Artist)-[:BORN_IN]->(:Place)-[:IN_COUNTRY]->(bk:Country)
MATCH (a)-[:DIED_IN]->(:Place)-[:IN_COUNTRY]->(dk:Country)
MATCH (a)-[:CREATED]->(:Artwork)-[:IN_COLLECTION]->(:Collection)
-[:LOCATED_IN]->(:Place)-[:IN_COUNTRY]->(k:Country)
WHERE k <> bk AND k <> dk
RETURN a.name AS artist, collect(DISTINCT k.name) AS foreign_countries

// Q34: Committenti multi-movimento
MATCH (p:Patron)-[:COMMISSIONED]->(:Artwork)<-[:CREATED]-(a:Artist)
-[:ASSOCIATED_WITH]->(m:Movement)
WITH p, collect(DISTINCT m.name) AS movements
WHERE size(movements) >= 2
RETURN p.name AS patron, movements

// Q35: Classifica dei paesi per movimenti ospitati
MATCH (k:Country)<-[:IN_COUNTRY]-(:Place)<-[:LOCATED_IN]-(:Collection)
<-[:IN_COLLECTION]-(:Artwork)-[:CLASSIFIED_IN]->(m:Movement)
RETURN k.name AS country, count(DISTINCT m) AS movements
ORDER BY movements DESC, country

```

L'implementazione delle query per la versione V2 deriva da quella della versione V1, mediante l'impiego di una pipeline dinamica di traduzione (espressione regolare *_reify*). Questo approccio converte a runtime le relazioni interessate (*P195*, *P1066*, *P737* e *P88*) in pattern a due salti che attraversano i nodi *statement*. Ad esempio, una ricerca del tipo *-[:STUDENT_OF]->* viene traslata in *-[:HAS_APPRENTICESHIP_STATEMENT]->(:ApprenticeshipStatement)-[:TO_MASTER]->*.

Tuttavia, tre query (Q16, Q23, Q32) necessitano di una riscrittura, poiché la reificazione impatta rispettivamente sul posizionamento dei qualificatori e sulla logica di conteggio dei cammini a lunghezza variabile. Di seguito il codice Cypher per queste tre query specifiche:

```

// Q16 (V2): Proprieta estratte dal CollectionStatement anziche dall'arco
MATCH (:Artist {name: $artist})-[:CREATED]->(w:Artwork)
-[:HAS_COLLECTION_STATEMENT]->(m:CollectionStatement)
-[:TO_COLLECTION]->(c:Collection)
RETURN w.name AS title, c.name AS collection,
m.since AS since, m.until AS until, m.inventory_number AS inventory_number

// Q23 (V2): Path su nodi intermedi (limite raddoppiato a 8 archi attraversati)
MATCH path = (a:Artist)-[:HAS_INFLUENCE_STATEMENT|TO_INFLUENCER*2..8]->
(:Artist {name: $influencer})
WITH path, [n IN nodes(path) WHERE n:Artist | n] AS artists
WHERE size(artists) - 1 <= 4
RETURN [n IN artists | n.name] AS influence_line, size(artists) - 1 AS hops

// Q32 (V2): Computo della distanza logica tra nodi di dominio (ignorando i nodi statement)
MATCH path = (:Artist {name: $artist})
-[:CREATED|HAS_INFLUENCE_STATEMENT|TO_INFLUENCER|
HAS_APPRENTICESHIP_STATEMENT|TO_MASTER*1..8]-
(:Artist {name: $artist2})
WITH path, [n IN nodes(path) WHERE n:Entity | n] AS domain_nodes
WHERE size(domain_nodes) - 1 <= 4
RETURN count(path) AS paths, min(size(domain_nodes) - 1) AS shortest

```

A differenza delle strutture a property graph, il modello RDF conserva la granularità a triple del dato originario e la molteplicità nativa dei valori, che le query devono gestire esplicitamente. Il grafo interrogato è quello finalizzato, sul quale è già stata applicata la selezione linguistica descritta nella Sezione 3.3: la coesistenza di letterali in più lingue permane unicamente nei dump grezzi. Di seguito vengono riportate le implementazioni in linguaggio

SPARQL delle 35 query, con la sola Q32 presentata in forma schematica poiché i rami a profondità 2–4 sono generati proceduralmente a runtime. Per gestire la cardinalità multipla tipica del modello RDF e garantire l'esatta equità dei risultati rispetto all'implementazione Cypher, le interrogazioni impiegano blocchi di *optional matching* ed estrazioni lessicografiche (tramite l'aggregatore *MIN*), includendo l'unrolling procedurale per superare i limiti strutturali del linguaggio SPARQL 1.1 sulle navigazioni di cammini a lunghezza limitata (Q23 e Q32).

```
PREFIX wdt: <http://www.wikidata.org/prop/direct/>
PREFIX ex: <https://artgraph.univpm.example/ontology#>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
PREFIX xsd: <http://www.w3.org/2001/XMLSchema#>

# Q01. Recuperare le informazioni anagrafiche di un artista.
SELECT ?artist ?born ?birth_place ?died ?death_place ?ulan WHERE {
  ?a a ex:Artist .
  FILTER EXISTS { ?a rdfs:label ?a_match . FILTER(STR(?a_match) = "$artist") }
  OPTIONAL { ?a rdfs:label ?artist }
  OPTIONAL { SELECT ?a (MIN(STR(?born_v)) AS ?born_raw) WHERE { ?a wdt:P569 ?born_v } GROUP BY ?a }
  BIND(SUBSTR(?born_raw,1,10) AS ?born)
  OPTIONAL { SELECT ?a (MIN(STR(?died_v)) AS ?died_raw) WHERE { ?a wdt:P570 ?died_v } GROUP BY ?a }
  BIND(SUBSTR(?died_raw,1,10) AS ?died)
  OPTIONAL { SELECT ?a (MIN(STR(?ulan_x_v)) AS ?ulan_x_raw) WHERE { ?a wdt:P245 ?ulan_x_v } GROUP BY ?a }
  BIND(?ulan_x_raw AS ?ulan)
  OPTIONAL { ?a ex:bornInCity ?bp . OPTIONAL { ?bp rdfs:label ?birth_place } }
  OPTIONAL { ?a ex:diedInCity ?dp . OPTIONAL { ?dp rdfs:label ?death_place } }
}

# Q02. Ottenere le informazioni di una certa opera d'arte.
SELECT ?title ?height_m ?width_m ?year ?image WHERE {
  ?w a ex:Artwork .
  FILTER EXISTS { ?w rdfs:label ?w_match . FILTER(STR(?w_match) = "$artwork") }
  OPTIONAL { ?w rdfs:label ?title }
  OPTIONAL { SELECT ?w (MIN(STR(?hm_v)) AS ?hm_raw) WHERE { ?w ex:heightM ?hm_v } GROUP BY ?w }
  BIND(xsd:decimal(?hm_raw) AS ?height_m)
  OPTIONAL { SELECT ?w (MIN(STR(?wm_v)) AS ?wm_raw) WHERE { ?w ex:widthM ?wm_v } GROUP BY ?w }
  BIND(xsd:decimal(?wm_raw) AS ?width_m)
  OPTIONAL { SELECT ?w (MIN(STR(?year_v)) AS ?year_raw) WHERE { ?w wdt:P571 ?year_v } GROUP BY ?w }
  BIND(xsd:integer(SUBSTR(?year_raw,1,4)) AS ?year)
  OPTIONAL { SELECT ?w (MIN(STR(?imgA_v)) AS ?imgA_raw) WHERE { ?w ex:imageUrl ?imgA_v } GROUP BY ?w }
  OPTIONAL { SELECT ?w (MIN(STR(?imgB_v)) AS ?imgB_raw) WHERE { ?w wdt:P18 ?imgB_v } GROUP BY ?w }
  BIND(COALESCE(?imgA_raw, ?imgB_raw) AS ?image)
}

# Q03. Elencare i materiali di cui è composta una determinata opera d'arte.
SELECT ?material WHERE {
  ?w a ex:Artwork .
  FILTER EXISTS { ?w rdfs:label ?w_match . FILTER(STR(?w_match) = "$artwork") }
  ?w wdt:P186 ?m . ?m a ex:Material .
  OPTIONAL { ?m rdfs:label ?material }
}

# Q04. Trovare per una determinata collezione in quale città si trova e le sue coordinate.
SELECT ?city ?coordinates WHERE {
  ?c a ex:Collection .
  FILTER EXISTS { ?c rdfs:label ?c_match . FILTER(STR(?c_match) = "$collection") }
  ?c ex:locatedInCity ?p . ?p a ex:Place .
  OPTIONAL { ?p rdfs:label ?city }
  OPTIONAL { SELECT ?p (MIN(STR(?coord_v)) AS ?coord_raw) WHERE { ?p wdt:P625 ?coord_v } GROUP BY ?p }
  BIND(?coord_raw AS ?coordinates)
}

# Q05. Elencare i dipinti di una determinata corrente artistica...
SELECT ?title ?year WHERE {
  ?w wdt:P135 ?m . ?w a ex:Artwork .
  ?m a ex:Movement .
  FILTER EXISTS { ?m rdfs:label ?m_match . FILTER(STR(?m_match) = "$movement") }
```

```

OPTIONAL { ?w rdfs:label ?title }
OPTIONAL { SELECT ?w (MIN(STR(?year_v)) AS ?year_raw) WHERE { ?w wdt:P571 ?year_v } GROUP BY ?w }
BIND(xsd:integer(SUBSTR(?year_raw,1,4)) AS ?year)
}

# Q06. Trovare gli artisti influenzati da un certo maestro.
SELECT ?artist WHERE {
  ?x wdt:P737 ?i . ?x a ex:Artist .
  ?i a ex:Artist .
  FILTER EXISTS { ?i rdfs:label ?i_match . FILTER(STR(?i_match) = "$influencer") }
  OPTIONAL { ?x rdfs:label ?artist }
}

# Q07. Elencare le opere di un artista e i paesi in cui sono conservate.
SELECT DISTINCT ?title ?country WHERE {
  ?a a ex:Artist .
  FILTER EXISTS { ?a rdfs:label ?a_match . FILTER(STR(?a_match) = "$artist") }
  ?w wdt:P170 ?a . ?w a ex:Artwork .
  ?w wdt:P195 ?c . ?c a ex:Collection .
  ?c ex:locatedInCity ?p . ?p a ex:Place .
  ?p wdt:P17 ?k . ?k a ex:Country .
  OPTIONAL { ?w rdfs:label ?title }
  OPTIONAL { ?k rdfs:label ?country }
}

# Q08. Trovare i maestri di un artista e i movimenti artistici...
SELECT ?master (GROUP_CONCAT(DISTINCT COALESCE(?mv_name, ""); SEPARATOR=" ") AS ?movements) WHERE {
  ?a a ex:Artist .
  FILTER EXISTS { ?a rdfs:label ?a_match . FILTER(STR(?a_match) = "$artist") }
  ?a wdt:P1066 ?m . ?m a ex:Artist .
  OPTIONAL { ?m rdfs:label ?master }
  OPTIONAL { ?m wdt:P135 ?mv . ?mv a ex:Movement . OPTIONAL { ?mv rdfs:label ?mv_name } }
} GROUP BY ?master

# Q09. Trovare le opere di un artista conservate nella stessa città di nascita.
SELECT ?title ?collection ?city WHERE {
  ?a a ex:Artist .
  FILTER EXISTS { ?a rdfs:label ?a_match . FILTER(STR(?a_match) = "$artist") }
  ?a ex:bornInCity ?p . ?p a ex:Place .
  ?c ex:locatedInCity ?p . ?c a ex:Collection .
  ?w wdt:P195 ?c . ?w a ex:Artwork .
  ?w wdt:P170 ?a .
  OPTIONAL { ?w rdfs:label ?title }
  OPTIONAL { ?c rdfs:label ?collection }
  OPTIONAL { ?p rdfs:label ?city }
}

# Q10. Trovare i movimenti artistici delle opere che raffigurano un determinato soggetto.
SELECT DISTINCT ?movement WHERE {
  ?s a ex:Subject .
  FILTER EXISTS { ?s rdfs:label ?s_match . FILTER(STR(?s_match) = "$subject") }
  ?w wdt:P180 ?s . ?w a ex:Artwork .
  ?w wdt:P135 ?m . ?m a ex:Movement .
  OPTIONAL { ?m rdfs:label ?movement }
}

# Q11. Elencare le opere di un certo genere create da artisti deceduti in una specifica città.
SELECT ?artist ?title WHERE {
  ?g a ex:Genre .
  FILTER EXISTS { ?g rdfs:label ?g_match . FILTER(STR(?g_match) = "$genre") }
  ?w wdt:P136 ?g . ?w a ex:Artwork .
  ?w wdt:P170 ?a . ?a a ex:Artist .
  ?a ex:diedInCity ?p . ?p a ex:Place .
  FILTER EXISTS { ?p rdfs:label ?p_match . FILTER(STR(?p_match) = "$city") }
  OPTIONAL { ?a rdfs:label ?artist }
  OPTIONAL { ?w rdfs:label ?title }
}

```

```

}

# Q12. Identificare i committenti che hanno commissionato opere a uno specifico artista.
SELECT ?patron ?title WHERE {
  ?w wdt:P170 ?a . ?a a ex:Artist .
  FILTER EXISTS { ?a rdfs:label ?a_match . FILTER(STR(?a_match) = "$commissioned_artist") }
  ?w a ex:Artwork .
  ?w wdt:P88 ?p . ?p a ex:Patron .
  OPTIONAL { ?p rdfs:label ?patron }
  OPTIONAL { ?w rdfs:label ?title }
}

# Q13. Trovare altri dipinti che raffigurano lo stesso soggetto...
SELECT DISTINCT ?title ?artist WHERE {
  ?w0 a ex:Artwork .
  FILTER EXISTS { ?w0 rdfs:label ?w0_match . FILTER(STR(?w0_match) = "$artwork") }
  ?w0 wdt:P180 ?s . ?s a ex:Subject .
  ?o wdt:P180 ?s . ?o a ex:Artwork .
  FILTER(?o != ?w0)
  ?o wdt:P170 ?a . ?a a ex:Artist .
  OPTIONAL { ?o rdfs:label ?title }
  OPTIONAL { ?a rdfs:label ?artist }
}

# Q14. Collezioni che conservano opere per materiale e movimento.
SELECT DISTINCT ?museum WHERE {
  ?w wdt:P186 ?m . ?m a ex:Material .
  FILTER EXISTS { ?m rdfs:label ?m_match . FILTER(STR(?m_match) = "$material") }
  ?w a ex:Artwork .
  ?w wdt:P135 ?mv . ?mv a ex:Movement .
  FILTER EXISTS { ?mv rdfs:label ?mv_match . FILTER(STR(?mv_match) = "$movement") }
  ?w wdt:P195 ?c . ?c a ex:Collection .
  OPTIONAL { ?c rdfs:label ?museum }
}

# Q15. Identificare gli artisti nati e morti nello stesso paese.
SELECT ?artist ?country (GROUP_CONCAT(DISTINCT COALESCE(?mv_name, ""); SEPARATOR=" ") AS ?movements) WHERE {
  ?a a ex:Artist .
  ?a ex:bornInCity ?bp . ?bp wdt:P17 ?k .
  ?a ex:diedInCity ?dp . ?dp wdt:P17 ?k .
  ?k a ex:Country .
  OPTIONAL { ?a rdfs:label ?artist }
  OPTIONAL { ?k rdfs:label ?country }
  OPTIONAL { ?a wdt:P135 ?mv . ?mv a ex:Movement . OPTIONAL { ?mv rdfs:label ?mv_name } }
} GROUP BY ?artist ?country ?bp ?dp

# Q16. Collezione e periodo di acquisizione/inventario delle opere di un artista.
SELECT ?title ?collection ?since ?until ?inventory_number WHERE {
  ?a a ex:Artist .
  FILTER EXISTS { ?a rdfs:label ?a_match . FILTER(STR(?a_match) = "$artist") }
  ?w wdt:P170 ?a . ?w a ex:Artwork .
  ?w wdt:P195 ?c . ?c a ex:Collection .
  OPTIONAL { ?w rdfs:label ?title }
  OPTIONAL { ?c rdfs:label ?collection }
  OPTIONAL {
    SELECT ?w ?c (MIN(STR(?since_v)) AS ?since_raw) WHERE {
      ?since_stmt ex:forSource ?w ; ex:forTarget ?c ; ex:since ?since_v .
    } GROUP BY ?w ?c
  }
}
BIND(SUBSTR(?since_raw,1,10) AS ?since)
OPTIONAL {
  SELECT ?w ?c (MIN(STR(?until_v)) AS ?until_raw) WHERE {
    ?until_stmt ex:forSource ?w ; ex:forTarget ?c ; ex:until ?until_v .
  } GROUP BY ?w ?c
}
BIND(SUBSTR(?until_raw,1,10) AS ?until)

```

```

OPTIONAL {
    SELECT ?w ?c (MIN(STR(?inv_v)) AS ?inventory_number) WHERE {
        ?inv_stmt ex:forSource ?w ; ex:forTarget ?c ; ex:inventoryNumber ?inv_v .
    } GROUP BY ?w ?c
}
}

# Q17. Materiali e generi pittorici per soggetto.
SELECT (GROUP_CONCAT(DISTINCT COALESCE(?m_name, ""); SEPARATOR=" ") AS ?materials)
       (GROUP_CONCAT(DISTINCT COALESCE(?g_name, ""); SEPARATOR=" ") AS ?genres) WHERE {
    ?s a ex:Subject .
    FILTER EXISTS { ?s rdfs:label ?s_match . FILTER(STR(?s_match) = "$subject") }
    ?w wdt:P180 ?s . ?w a ex:Artwork .
    OPTIONAL { ?w wdt:P186 ?m . ?m a ex:Material . OPTIONAL { ?m rdfs:label ?m_name } }
    OPTIONAL { ?w wdt:P136 ?g . ?g a ex:Genre . OPTIONAL { ?g rdfs:label ?g_name } }
}

# Q18. Movimenti degli artisti influenzati da un determinato pittore.
SELECT DISTINCT ?artist ?movement WHERE {
    ?x wdt:P737 ?i . ?i a ex:Artist .
    FILTER EXISTS { ?i rdfs:label ?i_match . FILTER(STR(?i_match) = "$influencer") }
    ?x a ex:Artist .
    ?x wdt:P135 ?m . ?m a ex:Movement .
    OPTIONAL { ?x rdfs:label ?artist }
    OPTIONAL { ?m rdfs:label ?movement }
}

# Q19. Collezioni di un paese con opere realizzate dagli allievi di un maestro.
SELECT DISTINCT ?museum ?student WHERE {
    ?s wdt:P1066 ?ma . ?ma a ex:Artist .
    FILTER EXISTS { ?ma rdfs:label ?ma_match . FILTER(STR(?ma_match) = "$master") }
    ?s a ex:Artist .
    ?w wdt:P170 ?s . ?w a ex:Artwork .
    ?w wdt:P195 ?c . ?c a ex:Collection .
    ?c ex:locatedInCity ?p . ?p a ex:Place .
    ?p wdt:P17 ?k . ?k a ex:Country .
    FILTER EXISTS { ?k rdfs:label ?k_match . FILTER(STR(?k_match) = "$country") }
    OPTIONAL { ?c rdfs:label ?museum }
    OPTIONAL { ?s rdfs:label ?student }
}

# Q20. Opere superiori a una certa altezza o larghezza.
SELECT ?title ?height_m ?width_m ?artist ?museum WHERE {
    ?w wdt:P170 ?a . ?w a ex:Artwork . ?a a ex:Artist .
    OPTIONAL { SELECT ?w (MIN(STR(?hm_v)) AS ?hm_raw) WHERE { ?w ex:heightM ?hm_v } GROUP BY ?w }
    BIND(xsd:decimal(?hm_raw) AS ?height_m)
    OPTIONAL { SELECT ?w (MIN(STR(?wm_v)) AS ?wm_raw) WHERE { ?w ex:widthM ?wm_v } GROUP BY ?w }
    BIND(xsd:decimal(?wm_raw) AS ?width_m)
    FILTER((BOUND(?height_m) && ?height_m >= $min_size_m) ||
           (BOUND(?width_m) && ?width_m >= $min_size_m))
    OPTIONAL { ?w rdfs:label ?title }
    OPTIONAL { ?a rdfs:label ?artist }
    OPTIONAL { ?w wdt:P195 ?c . ?c a ex:Collection . OPTIONAL { ?c rdfs:label ?museum } }
}

# Q21. Statistiche opere e area media, massima e minima per movimento.
SELECT ?movement (COUNT(?w) AS ?works)
               (ROUND(AVG(?area) * 10000) / 10000 AS ?avg_area_m2)
               (ROUND(MAX(?area) * 10000) / 10000 AS ?max_area_m2)
               (ROUND(MIN(?area) * 10000) / 10000 AS ?min_area_m2) WHERE {
    ?w wdt:P135 ?m . ?w a ex:Artwork . ?m a ex:Movement .
    OPTIONAL { SELECT ?w (MIN(STR(?hm_v)) AS ?hm_raw) WHERE { ?w ex:heightM ?hm_v } GROUP BY ?w }
    OPTIONAL { SELECT ?w (MIN(STR(?wm_v)) AS ?wm_raw) WHERE { ?w ex:widthM ?wm_v } GROUP BY ?w }
    FILTER(BOUND(?hm_raw) && BOUND(?wm_raw))
    BIND(ROUND(xsd:decimal(?hm_raw) * 10000) *
         ROUND(xsd:decimal(?wm_raw) * 10000) / 100000000 AS ?area)
}

```



```

    OPTIONAL { ?m rdfs:label ?movement }
} GROUP BY ?movement ORDER BY DESC(?works) ASC(?movement)

# Q22. I dieci soggetti più frequentemente raffigurati in opere di un movimento.
SELECT ?subject (COUNT(*) AS ?frequency) WHERE {
    ?m a ex:Movement .
    FILTER EXISTS { ?m rdfs:label ?m_match . FILTER(STR(?m_match) = "$movement") }
    ?w wdt:P135 ?m . ?w a ex:Artwork .
    ?w wdt:P180 ?s . ?s a ex:Subject .
    OPTIONAL { ?s rdfs:label ?subject }
} GROUP BY ?subject ORDER BY DESC(?frequency) ASC(?subject) LIMIT 10

# Q23. Ricostruire la catena di influenza (fino a 4 salti logici espliciti per aggirare i limiti di SPARQL 1.1).
SELECT ?a_name ?b1_name ?b2_name ?b3_name ?target_name ?hops WHERE {
    { ?a wdt:P737 ?t . BIND(1 AS ?hops) }
    UNION
    { ?a wdt:P737 ?b1 . ?b1 a ex:Artist . OPTIONAL { ?b1 rdfs:label ?b1_name }
      ?b1 wdt:P737 ?t . BIND(2 AS ?hops) }
    UNION
    { ?a wdt:P737 ?b1 . ?b1 a ex:Artist . OPTIONAL { ?b1 rdfs:label ?b1_name }
      ?b1 wdt:P737 ?b2 . ?b2 a ex:Artist . OPTIONAL { ?b2 rdfs:label ?b2_name }
      ?b2 wdt:P737 ?t . BIND(3 AS ?hops) }
    UNION
    { ?a wdt:P737 ?b1 . ?b1 a ex:Artist . OPTIONAL { ?b1 rdfs:label ?b1_name }
      ?b1 wdt:P737 ?b2 . ?b2 a ex:Artist . OPTIONAL { ?b2 rdfs:label ?b2_name }
      ?b2 wdt:P737 ?b3 . ?b3 a ex:Artist . OPTIONAL { ?b3 rdfs:label ?b3_name }
      ?b3 wdt:P737 ?t . BIND(4 AS ?hops) }
    ?a a ex:Artist .
    OPTIONAL { ?a rdfs:label ?a_name }
    ?t a ex:Artist .
    FILTER EXISTS { ?t rdfs:label ?t_match . FILTER(STR(?t_match) = "$influencer") }
    OPTIONAL { ?t rdfs:label ?target_name }
}

# Q24. Ordinare i committenti per numero di opere.
SELECT ?patron (COUNT(DISTINCT ?w) AS ?works)
    (GROUP_CONCAT(DISTINCT COALESCE(?a_name, ""); SEPARATOR=" ") AS ?artists) WHERE {
    ?w wdt:P88 ?p . ?p a ex:Patron . ?w a ex:Artwork .
    ?w wdt:P170 ?a . ?a a ex:Artist .
    OPTIONAL { ?p rdfs:label ?patron }
    OPTIONAL { ?a rdfs:label ?a_name }
} GROUP BY ?patron ORDER BY DESC(?works) ASC(?patron) LIMIT 5

# Q25. Coppie di artisti nati nella stessa città e stesso movimento.
SELECT DISTINCT ?artist1 ?artist2 ?city ?movement WHERE {
    ?a1 a ex:Artist . ?a2 a ex:Artist .
    ?a1 ex:bornInCity ?p . ?a2 ex:bornInCity ?p . ?p a ex:Place .
    ?a1 wdt:P135 ?m . ?a2 wdt:P135 ?m . ?m a ex:Movement .
    FILTER(STR(?a1) < STR(?a2))
    OPTIONAL { ?a1 rdfs:label ?artist1 }
    OPTIONAL { ?a2 rdfs:label ?artist2 }
    OPTIONAL { ?p rdfs:label ?city }
    OPTIONAL { ?m rdfs:label ?movement }
}

# Q26. Materiali più utilizzati da artisti nati in un determinato paese.
SELECT ?material (COUNT(*) AS ?uses) WHERE {
    ?a ex:bornInCity ?p . ?a a ex:Artist .
    ?p wdt:P17 ?k . ?k a ex:Country .
    FILTER EXISTS { ?k rdfs:label ?k_match . FILTER(STR(?k_match) = "$country") }
    ?w wdt:P170 ?a . ?w a ex:Artwork .
    ?w wdt:P186 ?m . ?m a ex:Material .
    OPTIONAL { ?m rdfs:label ?material }
} GROUP BY ?material ORDER BY DESC(?uses) ASC(?material)

```

```

# Q27. Dipinti con larghezza significativamente maggiore dell'altezza.
SELECT ?title ?width_m ?height_m ?artist ?museum WHERE {
  ?w wdt:P170 ?a . ?w a ex:Artwork . ?a a ex:Artist .
  OPTIONAL { SELECT ?w (MIN(STR(?hm_v)) AS ?hm_raw) WHERE { ?w ex:heightM ?hm_v } GROUP BY ?w }
  OPTIONAL { SELECT ?w (MIN(STR(?wm_v)) AS ?wm_raw) WHERE { ?w ex:widthM ?wm_v } GROUP BY ?w }
  FILTER(BOUND(?hm_raw) && BOUND(?wm_raw))
  BIND(xsd:decimal(?hm_raw) AS ?height_m)
  BIND(xsd:decimal(?wm_raw) AS ?width_m)
  FILTER(ROUND(?width_m * 10000) >= ROUND(?height_m * $ratio * 10000))
  OPTIONAL { ?w rdfs:label ?title }
  OPTIONAL { ?a rdfs:label ?artist }
  OPTIONAL { ?w wdt:P195 ?c . ?c a ex:Collection . OPTIONAL { ?c rdfs:label ?museum } }
}

# Q28. Allievi di un maestro e qualificatori temporali.
SELECT ?student ?since ?until WHERE {
  ?s wdt:P1066 ?ma . ?s a ex:Artist .
  ?ma a ex:Artist .
  FILTER EXISTS { ?ma rdfs:label ?ma_match . FILTER(STR(?ma_match) = "$master") }
  OPTIONAL { ?s rdfs:label ?student }
  OPTIONAL {
    SELECT ?s ?ma (MIN(STR(?since_v)) AS ?since_raw) WHERE {
      ?since_stmt ex:forSource ?s ; ex:forTarget ?ma ; ex:since ?since_v .
    } GROUP BY ?s ?ma
  }
  BIND(SUBSTR(?since_raw,1,10) AS ?since)
  OPTIONAL {
    SELECT ?s ?ma (MIN(STR(?until_v)) AS ?until_raw) WHERE {
      ?until_stmt ex:forSource ?s ; ex:forTarget ?ma ; ex:until ?until_v .
    } GROUP BY ?s ?ma
  }
  BIND(SUBSTR(?until_raw,1,10) AS ?until)
}

# Q29. Allievi associati a movimenti diversi da quelli del maestro.
SELECT ?student (GROUP_CONCAT(DISTINCT COALESCE(?sm_name, ""); SEPARATOR=" ") AS ?student_movements)
      ?master (GROUP_CONCAT(DISTINCT COALESCE(?mm_name, ""); SEPARATOR=" ") AS ?master_movements) WHERE {
  ?s wdt:P1066 ?m . ?s a ex:Artist . ?m a ex:Artist .
  OPTIONAL { ?s rdfs:label ?student }
  OPTIONAL { ?m rdfs:label ?master }
  OPTIONAL { ?s wdt:P135 ?sm . ?sm a ex:Movement . OPTIONAL { ?sm rdfs:label ?sm_name } }
  OPTIONAL { ?m wdt:P135 ?mm . ?mm a ex:Movement . OPTIONAL { ?mm rdfs:label ?mm_name } }
} GROUP BY ?student ?master

# Q30. Quante opere di ciascun genere pittorico create per decennio.
SELECT ?decade ?genre (COUNT(*) AS ?works) WHERE {
  ?w wdt:P136 ?g . ?w a ex:Artwork . ?g a ex:Genre .
  OPTIONAL { SELECT ?w (MIN(STR(?inc_v)) AS ?inc_raw) WHERE { ?w wdt:P571 ?inc_v } GROUP BY ?w }
  FILTER(BOUND(?inc_raw))
  BIND(xsd:integer(SUBSTR(?inc_raw,1,4)) AS ?y)
  BIND(xsd:integer(FLOOR(?y/10)) * 10 AS ?decade)
  OPTIONAL { ?g rdfs:label ?genre }
} GROUP BY ?decade ?genre ORDER BY ?decade DESC(?works) ASC(?genre)

# Q31. Coppie di artisti influenzati da almeno 2 stessi predecessori.
SELECT ?artist1 ?artist2 (GROUP_CONCAT(DISTINCT COALESCE(?x_name, ""); SEPARATOR=" ") AS ?shared) WHERE {
  ?a1 a ex:Artist . ?a2 a ex:Artist . ?x a ex:Artist .
  ?a1 wdt:P737 ?x . ?a2 wdt:P737 ?x .
  FILTER(STR(?a1) < STR(?a2))
  OPTIONAL { ?a1 rdfs:label ?artist1 }
  OPTIONAL { ?a2 rdfs:label ?artist2 }
  OPTIONAL { ?x rdfs:label ?x_name }
} GROUP BY ?artist1 ?artist2
HAVING(COUNT(DISTINCT ?x) >= 2)

# Q32. Percorsi e distanza minima tra due artisti generati esplicitamente (1..4 hops).

```

```

# NOTA: Per la natura limitata di SPARQL 1.1 sui percorsi nidificati, l'algoritmo
# srotola (unrolling) la ricerca in Python e inietta le triple complete
# tramite UNION esplicite vincolando i rami esplorativi per simulare
# la Index-Free Adjacency propria dei linguaggi a grafo.
SELECT (COUNT(*) AS ?paths) (MIN(?hops) AS ?shortest) WHERE {
  {
    { <IRI_Artist1> wdt:P170 <IRI_Artist2> . <IRI_Artist1> a ex:Artwork . <IRI_Artist2> a ex:Artist . }
    UNION { <IRI_Artist2> wdt:P170 <IRI_Artist1> . <IRI_Artist2> a ex:Artwork . <IRI_Artist1> a ex:Artist . }
  }
  UNION { <IRI_Artist1> wdt:P737 <IRI_Artist2> . <IRI_Artist1> a ex:Artist . <IRI_Artist2> a ex:Artist . }
  UNION { <IRI_Artist2> wdt:P737 <IRI_Artist1> . <IRI_Artist2> a ex:Artist . <IRI_Artist1> a ex:Artist . }
  UNION { <IRI_Artist1> wdt:P1066 <IRI_Artist2> . <IRI_Artist1> a ex:Artist . <IRI_Artist2> a ex:Artist . }
  UNION { <IRI_Artist2> wdt:P1066 <IRI_Artist1> . <IRI_Artist2> a ex:Artist . <IRI_Artist1> a ex:Artist . }
  BIND(1 AS ?hops)
}
UNION
{
  # Blocco generato proceduralmente a runtime per l'esplorazione a 2, 3 e 4 salti
  # (omette le triple intermedie identiche ai rami del salto 1 per leggibilità)
  BIND(2 AS ?hops)
}
}

# Q33. Artisti con almeno un'opera in un paese diverso da nascita o morte.
SELECT ?artist (GROUP_CONCAT(DISTINCT COALESCE(?k_name, ""); SEPARATOR=" ") AS ?foreign_countries) WHERE {
  ?a a ex:Artist .
  ?a ex:bornInCity ?bp . ?bp wdt:P17 ?bk . ?bk a ex:Country .
  ?a ex:diedInCity ?dp . ?dp wdt:P17 ?dk . ?dk a ex:Country .
  ?w wdt:P170 ?a . ?w a ex:Artwork .
  ?w wdt:P195 ?c . ?c a ex:Collection .
  ?c ex:locatedInCity ?p . ?p a ex:Place .
  ?p wdt:P17 ?k . ?k a ex:Country .
  FILTER(?k != ?bk && ?k != ?dk)
  OPTIONAL { ?a rdfs:label ?artist }
  OPTIONAL { ?k rdfs:label ?k_name }
} GROUP BY ?artist

# Q34. Committenti per artisti di almeno 2 movimenti diversi.
SELECT ?patron (GROUP_CONCAT(DISTINCT COALESCE(?m_name, ""); SEPARATOR=" ") AS ?movements) WHERE {
  ?w wdt:P88 ?p . ?p a ex:Patron . ?w a ex:Artwork .
  ?w wdt:P170 ?a . ?a a ex:Artist .
  ?a wdt:P135 ?m . ?m a ex:Movement .
  OPTIONAL { ?p rdfs:label ?patron }
  OPTIONAL { ?m rdfs:label ?m_name }
} GROUP BY ?patron
HAVING(COUNT(DISTINCT ?m) >= 2)

# Q35. Classificare i paesi in base al numero di movimenti artistici ospitati.
SELECT ?country (COUNT(DISTINCT ?m) AS ?movements) WHERE {
  ?c ex:locatedInCity ?p . ?c a ex:Collection . ?p a ex:Place .
  ?p wdt:P17 ?k . ?k a ex:Country .
  ?w wdt:P195 ?c . ?w a ex:Artwork .
  ?w wdt:P135 ?m . ?m a ex:Movement .
  OPTIONAL { ?k rdfs:label ?country }
} GROUP BY ?country ORDER BY DESC(?movements) ASC(?country)

```

5.2 Metodologia di Test e Risultati Prestazionali

Il benchmark garantisce l'equità del confronto resettando e ripopolando i database dal file sorgente *artgraph.ttl* prima di ogni sessione, cosicché le tre configurazioni misurino sempre il medesimo contenuto logico. Il reset

ripristina il contenuto del database, ma non garantisce un identico layout fisico dei record: come documentato nella Sezione 5.3, è proprio questa differenza di disposizione a generare una parte della varianza osservata. Ogni query viene testata in tre fasi:

1. **Warm-up:** Esecuzione preliminare non cronometrata per l’ottimizzazione dei piani di query, il popolamento delle cache e la validazione logica dei risultati.
2. **Esecuzioni cronometrate:** 10 iterazioni consecutive per misurare la latenza fisica end-to-end (dall’invio della richiesta alla completa ricezione e decodifica delle righe da parte del client). Dalla distribuzione dei tempi si estraggono il valore minimo, la mediana (p50) e il novantacinquesimo percentile (p95).
3. **Profiling fisico:** Esecuzione finale tramite clausola *PROFILE* per quantificare gli accessi logici (*database hits*) effettuati dal motore di archiviazione.

L’interazione con i database avviene tramite le interfacce con cui i due motori vengono realmente utilizzati: driver Bolt (TCP) per Neo4j ed endpoint SPARQL (HTTP) per Oxigraph, entrambi eseguiti come processi server distinti e ricaricati da zero prima di ogni sessione. La misura include così l’overhead di rete e di serializzazione tipico delle architetture reali; in particolare Oxigraph non viene interrogato attraverso la libreria in-process *pyoxigraph*, impiegata unicamente nella fase di costruzione del grafo, che gli attribuirebbe un vantaggio non confrontabile. I due protocolli di trasporto non sono comunque equivalenti, e una quota del divario osservato è imputabile a tale differenza più che al modello dei dati.

Poiché le metriche assolute si sono rivelate poco stabili fra cicli di ricaricamento (Sezione 5.3), la Tabella 5.2 riporta l’intervallo osservato su quattro cicli indipendenti di *wipe-and-reload* anziché il valore di una singola sessione.

Misura	V1	V2	RDF/SPARQL
Nodi (LPG) / Triple (RDF)	2 103	2 855	18 958
Relazioni	6 294	7 046	—
Tempo di caricamento (s)	3,8–7,0	4,7–5,5	0,03–0,05
Somma dei tempi mediani, 35 query (ms)	105–205	86–141	1 243–1 263
Accessi al database (PROFILE, somma)	98 500–117 465	110 081–125 637	n.d.
Rapporto accessi V2/V1		1,07–1,12	n.d.
Equivalenza dei result set	35/35	35/35	35/35

Tabella 5.2: Confronto sulle 35 query, a parità di contenuto informativo e di parametri di input. Gli intervalli sono misurati su quattro cicli indipendenti di ricaricamento. In **blu** il valore migliore delle sole righe che esprimono una prestazione; le righe di cardinalità e di equivalenza non ammettono un ordinamento di merito.

L’analisi dei tempi di esecuzione evidenzia un trade-off prestazionale tra l’architettura RDF/SPARQL e la soluzione Labeled Property Graph (LPG).

In fase di *data ingestion*, il *triplestore* RDF (Oxigraph) registra tempi di caricamento contenuti, dell’ordine dei centesimi di secondo. Una spiegazione coerente con questo comportamento risiede nella natura append-only e orientata alle asserzioni atomiche del motore RDF, che non richiede la strutturazione preliminare di tabelle di adiacenza né la compilazione di vincoli. Neo4j mostra invece latenze d’ingestione di alcuni secondi, plausibilmente riconducibili al costo di allocazione dei record fisici per nodi e archi, alla costruzione degli indici secondari e alla verifica a runtime dei vincoli di unicità sull’attributo *qid*. Il confronto non isola però il modello dei dati: il caricamento di Neo4j comprende indici e vincoli che nella configurazione RDF non hanno corrispettivo.

In fase di esecuzione del workload, la somma delle latenze mediane di Oxigraph risulta di un ordine di grandezza superiore rispetto a entrambe le configurazioni Neo4j, ed è l’unico scarto sufficientemente ampio da non essere

spiegabile con la variabilità fra cicli. Una spiegazione compatibile con le rispettive architetture riguarda il diverso modo in cui i due motori materializzano le relazioni e ne risolvono le composizioni: un triplestore risolve i pattern *multi-hop* come sequenze di join su indici delle triple, mentre Neo4j adotta l'*Index-Free Adjacency* [9], in cui le relazioni di un nodo sono raggiungibili senza ricorrere a un indice. Resta un'interpretazione: il protocollo adottato non discrimina il contributo del modello dei dati da quello del trasporto (HTTP contro Bolt) e delle rispettive implementazioni. La conclusione difendibile è dunque che, in questa implementazione e su questo workload, Oxigraph è risultato più lento di Neo4j, e non che il modello RDF sia intrinsecamente meno efficiente.

Il confronto fra le due topologie LPG, invece, non consente di stabilire una gerarchia sul piano della latenza: gli intervalli di V1 e V2 si sovrappongono ampiamente e il loro rapporto oscilla, fra i cicli misurati, da 0,69 a 1,05. Sul piano degli accessi logici il quadro è più netto e stabile: V2 ne richiede sistematicamente fra il 7% e il 12% in più di V1, come atteso per una topologia che sostituisce 752 archi diretti con altrettanti nodi intermedi e 1.504 archi. Il costo della reificazione si manifesta quindi come lavoro aggiuntivo dello storage engine, misurabile in modo riproducibile, più che come penalità di tempo osservabile a questa scala di dati.

5.3 Analisi dell'Instabilità di Profiling

Ripetendo l'intera campagna su quattro cicli indipendenti di *wipe-and-reload*, a parità di dataset, di codice di ingestione e di query, gli accessi logici complessivi variano in modo consistente: la somma sulle 35 query oscilla fra 98.500 e 117.465 per V1 e fra 110.081 e 125.637 per V2, corrispondenti a un'escursione rispettivamente del 17% e del 13% attorno alla media. La variabilità non riguarda dunque poche query ad alta densità relazionale, ma l'aggregato complessivo.

Questa instabilità non si trasmette al confronto fra le due topologie: il rapporto fra gli accessi di V2 e quelli di V1 resta compreso fra 1,07 e 1,12 in tutti i cicli, con un'escursione di appena cinque punti percentuali. La grandezza assoluta è quindi poco riproducibile, mentre il rapporto fra le due configurazioni misurate nelle stesse condizioni è stabile: è su quest'ultimo che vengono fondate le conclusioni della sezione precedente.

Quanto all'origine del fenomeno, i piani di esecuzione compilati dall'ottimizzatore restano identici operatore per operatore fra un ciclo e l'altro, e i result set sono verificati equivalenti; la differenza va quindi ricercata nella disposizione fisica dei record anziché nella strategia di esecuzione. Un meccanismo coerente con questa osservazione, e documentato nell'architettura dello storage engine di Neo4j, è il seguente: la cancellazione tramite *DETACH DELETE* marca gli identificatori interni di nodi e archi come riutilizzabili nelle *free-list*, e il successivo caricamento batch li riassegna in un ordine che dipende dalla schedulazione delle transazioni. Poiché le relazioni afferenti a un nodo sono organizzate in catene di adiacenza, la posizione di un arco all'interno della catena può variare fra un caricamento e l'altro, e con essa il numero di record che un operatore di espansione deve attraversare prima di individuarlo, a parità di risultato prodotto.

L'esperimento condotto osserva la variazione e ne verifica la compatibilità con questa spiegazione, ma non la dimostra: isolare il contributo della riallocazione degli identificatori da quello della gestione della memoria richiederebbe una strumentazione a livello di storage engine che esula dal perimetro del progetto.

La conseguenza metodologica è comunque netta: una singola campagna di benchmark, per quanto ripetuta su molte iterazioni cronometrate, non caratterizza in modo affidabile le prestazioni assolute del motore, poiché le iterazioni condividono il medesimo layout fisico e ne ereditano il bias. Per confrontare scelte di modellazione occorre ripetere il ciclo di caricamento e ragionare su rapporti fra configurazioni, come fatto in questa sede.

6. Disegno del Benchmark Comparativo RAG vs GraphRAG

Il confronto oppone un'architettura RAG documentale a diverse implementazioni di GraphRAG. Il modello di linguaggio, il prompt di sintesi e il budget massimo di contesto sono mantenuti costanti fra le sette configurazioni, così da isolare la pipeline di recupero come unica variabile indipendente.

L'esperimento ha una natura circoscritta. Le domande di test sono generate a partire dai cammini del grafo (Sezione 6.2), ossia dalla stessa rappresentazione su cui operano le pipeline GraphRAG: si tratta quindi di un *benchmark multi-hop derivato da grafo*, non di una comparazione del tutto neutrale fra i due paradigmi. Analogamente, il grafo deriva da Wikidata mentre il corpus documentale deriva da Wikipedia: le due fonti coprono le medesime entità e sottostanno al medesimo limite di contesto, ma non veicolano un insieme identico di fatti. Nel seguito si parlerà pertanto di allineamento sui domini di entità e di identico limite massimo di contesto, e non di parità di evidenza informativa. La Sezione 8.2 discute sistematicamente le conseguenze di queste scelte.

6.1 Costruzione del Corpus Testuale Allineato

Per rendere confrontabile il recupero vettoriale con le pipeline GraphRAG, è stata implementata una strategia di allineamento che riduce le asimmetrie nel dominio di conoscenza accessibile ai diversi sistemi. Per mitigare il bias derivante dall'assenza di dati contestuali nel paradigma documentale, il corpus non strutturato non è limitato alle sole entità primarie (artisti e opere), ma riflette l'intera topologia dei nodi presenti in ArtGraph. Il processo di acquisizione esegue l'estrazione dei testi dalle voci Wikipedia in lingua inglese per tutte le classi definite in *settings.CORPUS_ENTITY_GROUPS*. Questo garantisce che il RAG documentale operi su una base di conoscenza comprensiva di:

- **Entità Target:** dati biografici dei pittori (seed e referenziati) e metadati narrativi delle opere;
- **Nodi Relazionali:** informazioni contestuali su movimenti artistici, istituzioni museali, entità geografiche (città e nazioni), materiali e generi pittorici.

Le classi *Subject* e *Patron* non costituiscono gruppi di raccolta autonomi: le entità che ricoprono anche tali ruoli entrano nel corpus in quanto già censite in uno degli otto gruppi elencati. Il processo ha raccolto 923 articoli, per complessivi 17,2 milioni di caratteri di testo.

Strategia di segmentazione (*chunking*)

La scelta della strategia di segmentazione incide direttamente sulle prestazioni del recupero vettoriale e va quindi esplicitata come parte del disegno sperimentale. I documenti estratti (*articles.jsonl*) sono processati tramite un

algoritmo deterministico a finestra scorrevole, che integra vincoli di integrità sintattica affinché nessun frammento risulti troncato a metà di una frase. I parametri adottati sono i seguenti:

1. **Dimensione target e sovrapposizione:** ogni frammento tende a una lunghezza di 1.200 caratteri (*CHUNK_TARGET_CHARS*), con una sovrapposizione di 200 caratteri (*CHUNK_OVERLAP_CHARS*) fra segmenti consecutivi, in modo che un fatto a cavallo di due frammenti resti recuperabile da almeno uno dei due. I frammenti terminali inferiori a 200 caratteri (*CHUNK_MIN_CHARS*) vengono scartati in quanto privi di contesto sufficiente.
2. **Priorità di taglio:** entro una finestra di tolleranza a ridosso della dimensione target, il punto di separazione è scelto privilegiando nell'ordine i delimitatori di paragrafo (*\n\n*) e, in loro assenza, i confini di frase (espressione regolare *[.!?]\s*); solo come ultima istanza si applica il taglio netto alla soglia.
3. **Mappatura sui nodi:** ogni frammento (*chunks.jsonl*) è corredato dal metadato *qid* dell'entità di provenienza, il che rende tracciabile il legame fra evidenza testuale e nodo del Knowledge Graph.

La segmentazione così configurata produce 18.741 frammenti, di lunghezza media pari a 1.109 caratteri. Il valore di 1.200 caratteri rappresenta un compromesso: frammenti più brevi frazionerebbero i fatti biografici su più unità, riducendo la probabilità che una singola unità recuperata contenga la risposta completa; frammenti più lunghi diluirebbero il segnale semantico nel vettore di embedding, penalizzando la precisione della ricerca per similarità. I parametri non sono stati ottimizzati sulle domande del benchmark: sono fissati a priori e mantenuti costanti, così da non avvantaggiare la baseline vettoriale tramite *chunk engineering*.

Il modello di embedding impiegato è *jina-embeddings-v5-text-nano-retrieval*¹ [10], variante *nano* da 239 milioni di parametri specializzata per il recupero, servita localmente sulla medesima infrastruttura. Il modello produce vettori a 768 dimensioni (la dimensionalità piena, poiché il supporto Matryoshka consentirebbe di troncarli a 512, 256 o meno) e accetta sequenze fino a 8.192 token, ampiamente superiori alla lunghezza dei frammenti generati.

Il modello è addestrato in modo asimmetrico: domande e passaggi vanno codificati con marcatori distinti perché il vettore di una domanda si avvicini a quello del passaggio che la risolve. La configurazione adottata impiega i marcatori della famiglia E5 (*query*: per le domande, *passage*: per i frammenti), mentre la scheda del modello raccomanda per la variante *retrieval* i marcatori nativi *Query*: e *Document*:. I prefissi intervengono unicamente nella chiamata di embedding e non sono mai mostrati al modello che genera la risposta, lasciando invariato il testo del contesto.

Poiché la baseline vettoriale è il termine di paragone dell'intero esperimento, l'effetto di questa discrepanza è stato quantificato a posteriori con una verifica dedicata, ri-codificando con entrambe le convenzioni un medesimo insieme di 2.451 frammenti candidati e misurando su 25 quesiti stratificati la posizione del frammento contenente la risposta. I marcatori nativi risultano lievemente superiori: il frammento corretto compare in prima posizione in 15 casi su 25 contro 12, ed entro le prime cinque in 20 casi contro 17; la posizione mediana resta la prima per entrambe le convenzioni. La configurazione adottata penalizza dunque la baseline in misura contenuta e reale, ma di entità non paragonabile al divario riportato nel benchmark. Il limite è ripreso nella Sezione 8.2.

6.2 Generazione e Validazione delle Domande di Test

La valutazione delle pipeline di recupero è basata su un set di test composto da circa 100 domande, generate mediante una procedura automatizzata e controllata. L'obiettivo è quantificare l'accuratezza del sistema in funzione della profondità relazionale del quesito. Il processo di generazione è strutturato in due fasi sequenziali:

¹<https://huggingface.co/jinaai/jina-embeddings-v5-text-nano-retrieval>

1. **Generazione guidata dai cammini del grafo:** Le domande sono derivate per ispezione deterministica del grafo *artgraph.ttl*. Per ogni quesito, viene estratto il cammino logico di riferimento e memorizzato nel campo *Question.path* come sequenza di triple [*soggetto, predicato, oggetto*]. Tale struttura permette di classificare i test case in base alla distanza di *hop* necessaria per la risoluzione:
 - **Easy (1-hop):** Interrogazioni puntuali su attributi diretti o relazioni immediate (es. individuazione del materiale di un'opera o dell'anno di creazione).
 - **Intermediate (2-hop):** Quesiti che richiedono il passaggio attraverso un nodo intermedio di raccordo (es. identificazione della città di nascita di un autore partendo da una sua opera: *Artwork* → *Artist* → *Place*).
 - **Hard (cammini, 2–3 hop):** Quesiti di attraversamento, in cui la risposta si ottiene percorrendo una catena di relazioni omogenee anziché combinando attributi eterogenei. Vi rientrano le catene di apprendistato fra autori (allievo → maestro → maestro del maestro, 2 hop) e la localizzazione geografica di un'opera attraverso l'istituzione che la conserva (*Artwork* → *Collection* → *Place* → *Country*, 3 hop). La classe è definita dalla natura del cammino e non dalla sola profondità: la difficoltà per un sistema documentale risiede nel fatto che ciascun anello della catena risiede in una voce distinta, e che l'anello finale coincide con la voce esclusa dal corpus. Nessun quesito del dataset supera i 3 hop.
2. **Verifica di risolvibilità documentale (*Supporting Facts*):** Per rendere confrontabile il benchmark fra il modello vettoriale e quello strutturato, viene applicato un filtro di validazione ispirato ai *supporting facts* del dataset *HotpotQA* [2], in cui ogni quesito multi-hop è corredato dalle evidenze testuali che ne consentono la risoluzione. Un test case è considerato valido solo se soddisfa i seguenti vincoli di grounding documentale:
 - **Grounding lessicale:** I token identificativi della risposta corretta (*must_include*) devono essere presenti nel testo dei documenti Wikipedia associati ai nodi che compongono il cammino logico del quesito.
 - **Esclusione del bias di lookup:** È esclusa sistematicamente la voce Wikipedia corrispondente all'entità-risposta (es. se la risposta è "Paris", l'evidenza deve essere rintracciata negli articoli del pittore o del dipinto e non nella voce geografica di Parigi). Questo vincolo forza il sistema di retrieval a eseguire un effettivo concatenamento informativo tra più documenti, impedendo la risoluzione tramite semplice ricerca per parola chiave sul target.

6.3 Le 7 Pipeline di Retrieval a Confronto

Il benchmark valuta sette configurazioni di recupero informativo, operanti su un modello di linguaggio (LLM) invariante e soggette a un vincolo di serializzazione del contesto (*CONTEXT_CHAR_BUDGET*) fissato a 24.000 caratteri. Il limite è espresso in caratteri e non in token: è quindi una misura di occupazione testuale, non la finestra di contesto del modello, che resta ampiamente più capiente. Tale soglia è dimensionata perché i dieci frammenti del RAG vettoriale vi rientrino integralmente: il limite garantisce che nessuna pipeline sia penalizzata dal troncamento delle evidenze. Quanto contesto ciascuna pipeline effettivamente occupi entro tale soglia è invece uno dei risultati della misurazione. Le pipeline sono categorizzate in tre paradigmi distinti:

1. **RAG documentale:** costituisce la baseline vettoriale densa dell'esperimento, secondo lo schema *Retrieval-Augmented Generation* introdotto da Lewis et al. [1] e la formulazione a doppio encoder del recupero denso [11]. La domanda è convertita in un vettore tramite il modello di embedding configurato, quindi si esegue una ricerca per *cosine similarity* su PostgreSQL esteso con *pgvector* [12], estraendo i 10 frammenti più affini

(*RETRIEVAL_TOP_K = 10*), che vengono concatenati nel blocco di contesto. La pipeline non impiega re-ranking, decomposizione della domanda, interrogazione multipla né recupero iterativo: si tratta di una scelta deliberata, finalizzata a isolare la modalità con cui l'evidenza raggiunge il modello anziché l'ingegnerizzazione del recupero. Ne consegue che i risultati vanno riferiti a questa specifica baseline e non al paradigma RAG nel suo complesso.

2. **Graph_Query_V1 (Text-to-Query su Neo4j V1):** Il modello di linguaggio riceve all'interno del prompt di sistema la specifica completa dello schema logico del database Neo4j V1 (etichette dei nodi, proprietà e relazioni ammesse). In un'unica chiamata, il modello ha il compito di generare una query Cypher sintatticamente valida. Le righe restituite dalla query vengono formattate come testo ed inserite nel contesto finale. In caso di errore di sintassi o violazione semantica restituito dal database, il framework consente un numero massimo di tentativi di correzione controllati (*QUERY_MAX_RETRIES*) prima di decretare il fallimento del recupero.
3. **Graph_Query_V2 (Text-to-Query su Neo4j V2):** Segue la medesima architettura Text-to-Query della pipeline precedente, operando tuttavia sulla versione reificata del database (Neo4j V2). Lo schema fornito all'LLM riflette l'introduzione dei nodi statement intermedi per le relazioni di collezione, apprendistato, influenza e committenza, costringendo il modello di linguaggio a generare cammini Cypher a due salti logici per intercettare i medesimi fatti.
4. **Graph_Query_RDF (Text-to-Query su Oxigraph):** Implementa l'approccio Text-to-Query traducendo la domanda in linguaggio naturale in una query SPARQL [13] eseguita sul motore Oxigraph [14], interrogato come processo server locale. I prefissi del namespace ontologico di ArtGraph (*ex:*) vengono inseriti automaticamente prima dell'esecuzione del codice per ottimizzare la compattezza della query generata.
5. **Graph_Expand_V1 (Text-to-Expansion su Neo4j V1):** Questa strategia adotta l'approccio Text-to-Expansion. Anziché delegare all'LLM la stesura di codice di interrogazione nativo, esposta ad allucinazioni di schema, la pianificazione viene segmentata. Il modello analizza la domanda e lo schema per produrre un piano di espansione strutturato in formato JSON, specificando i nodi di partenza (*seeds*), le relazioni da percorrere e la profondità massima dell'esplorazione (*hops*). Ricevuto il piano, un motore deterministico scritto in Python esegue l'estrazione del sottografo in modo controllato, rispettando i limiti massimi configurati (*EXPAND_MAX_NODES* e *EXPAND_MAX_EDGES*). Il sottografo, arricchito con gli attributi scalari dei nodi seed, viene serializzato in formato di testo omogeneo e inviato al prompt finale.
6. **Graph_Expand_V2 (Text-to-Expansion su Neo4j V2):** Esegue l'espansione deterministica del piano JSON formulato dal modello sul database reificato Neo4j V2, gestendo la risoluzione delle relazioni attraverso l'attraversamento dei nodi intermedi di statement.
7. **Graph_Expand_RDF (Text-to-Expansion su Oxigraph):** Applica la medesima strategia di pianificazione ed espansione deterministica sul database a triple RDF, convertendo internamente il piano in percorsi a triple sequenziali ed esportando i fatti in un formato testuale di output equivalente a quello delle versioni LPG.

Nelle tre pipeline Text-to-Query la query raggiunge il database attraverso un server *Model Context Protocol* [15], avviato come processo figlio su trasporto *stdio* e configurato sul backend in esame: il server offre a Neo4j e a Oxigraph un'interfaccia unica e un unico punto in cui applicare il vincolo di sola lettura. Il modello non vi partecipa: non riceve alcuno schema di strumenti, non esegue quindi *tool calling* e non sceglie l'operazione da invocare. Lo schema del database è testo nel prompt di sistema, il modello restituisce il solo testo della query

e il framework vi invoca *run_query*; le altre due operazioni esposte dal server, *get_schema* e *search_nodes*, non vengono mai chiamate. Le pipeline Text-to-Expansion non transitano per il server.

Per ogni esecuzione, vengono acquisite metriche granulari relative all'accuratezza semantica (validata tramite *LLM-as-a-judge* [16]), alla latenza di risposta del motore di *retrieval* e alla densità di occupazione del contesto finale.

7. Analisi Sperimentale e Risultati RAG vs Graph-RAG

I risultati del benchmark comparativo sono analizzati sulle metriche di accuratezza, latenza e densità informativa dei due paradigmi di recupero, documentale e strutturato.

7.1 Configurazione dei Test e Metriche Utilizzate

La valutazione delle pipeline, come descritto nella sezione 6.2, è stata condotta su un dataset di 100 quesiti, validati preventivamente per garantire il grounding informativo all'interno del corpus Wikipedia. Il dataset si ripartisce in 60 quesiti *easy* (1 hop), 29 *intermediate* (2 hop) e 11 *hard* (cammini a 2 o 3 hop): la fascia più complessa è quindi poco numerosa, e le percentuali che la riguardano vanno lette con la cautela che tale cardinalità impone. Poiché la profondità massima richiesta dal dataset è di 3 hop, il limite *EXPAND_MAX_HOPS* imposto alle pipeline Text-to-Expansion è fissato al medesimo valore: un piano di espansione corretto può quindi sempre raggiungere l'entità-risposta, e i fallimenti osservati sono imputabili alla pianificazione e non al vincolo di profondità.

Il protocollo operativo ha impiegato tre varianti della famiglia Gemma 4, eseguite in locale su una workstation NVIDIA DGX Spark, equipaggiata con superchip GB10 Grace Blackwell e 128 GB di memoria unificata [17], e servite tramite endpoint OpenAI-compatibili:

- *gemma-4-26B-A4B-it*¹ [18], quantizzazione FP8 dinamica distribuita da Red Hat AI² [19];
- *gemma-4-E4B-it*³ [20];
- *gemma-4-E2B-it*⁴ [21].

Le tre varianti non costituiscono una scala densa ordinata per numero di parametri. La prima adotta un'architettura *Mixture-of-Experts*, in cui a fronte di 25,2 miliardi di parametri totali soltanto 3,8 miliardi risultano attivi per token, secondo la convenzione di denominazione *A4B*. Le varianti E4B ed E2B impiegano invece le *Per-Layer Embeddings*, che assegnano a ciascuno strato una tabella di embedding dedicata usata come semplice lookup: ne deriva una nozione di parametri *effective* (4,5 miliardi su 8 complessivi per E4B; 2,3 su 5,1 per E2B) sensibilmente inferiore al totale nominale. Le differenze osservate vanno pertanto attribuite alla capacità complessiva della variante impiegata, e non interpretate come effetto causale del solo conteggio parametrico.

Per ciascuna configurazione sono state eseguite tre repliche complete, allo scopo di misurare la variabilità fra esecuzioni; i valori riportati nelle tabelle sono medie sulle tre repliche. Il numero di repliche non è sufficiente a

¹<https://huggingface.co/google/gemma-4-26B-A4B-it>

²<https://huggingface.co/RedHatAI/gemma-4-26B-A4B-it-FP8-dynamic>

³<https://huggingface.co/unsloth/gemma-4-E4B-it>

⁴<https://huggingface.co/unsloth/gemma-4-E2B-it>

fondare test di ipotesi formali, e nel seguito non viene rivendicata alcuna significatività statistica. La deviazione standard fra repliche è comunque risultata trascurabile per l'accuratezza (nulla nella maggioranza delle configurazioni, con massimo di 0,6 punti percentuali), coerentemente con l'impiego di temperatura 0 e seed fissato: la variabilità residua si concentra nelle latenze, sensibili al carico della macchina di test.

Per isolare le performance della componente di *retrieval* dalla capacità di sintesi dei modelli, è stato imposto un limite di serializzazione del contesto (*CONTEXT_CHAR_BUDGET*) pari a 24.000 caratteri, dimensionato in modo che nessuna pipeline subisca troncamento delle evidenze.

Il sistema di valutazione adotta il paradigma LLM-as-a-judge [16], che impiega un modello di linguaggio come valutatore automatico in luogo di annotatori umani. Il giudice è il medesimo modello impiegato per la generazione all'interno di ciascuna sessione, interrogato con temperatura 0 e seed fissato, e riceve domanda, *gold answer* e risposta candidata, restituendo un verdetto binario espresso da un unico token (CORRECT / INCORRECT). La rubrica istruisce il giudice ad accettare differenze di formulazione, dettagli aggiuntivi corretti e ordinamenti diversi, e a rifiutare le risposte prive del fatto chiave, contraddittorie rispetto al riferimento o che dichiarano l'assenza di informazioni. Per rendere verificabile ogni decisione, il prompt integrale sottoposto al giudice e il verdetto restituito sono persistiti insieme a ciascun record di traccia.

Poiché il verdetto è affidato a un modello, è stato sottoposto a un controllo esterno indipendente. Le domande dispongono infatti di un'etichetta strutturata (*must_include*), che consente di derivare un secondo verdetto puramente lessicale e deterministico: applicato a posteriori a tutte le 6.300 risposte persistite, concorda con il giudice nel 96,2% dei casi. L'esame dei 240 disaccordi conferma che il giudice è il criterio più affidabile e va quindi mantenuto: esso riconosce equivalenze semantiche che il confronto fra stringhe non cattura ("*French Romantic artist*" per *Romanticism*), mentre il riscontro lessicale produce falsi positivi accettando risposte in cui il token compare solo perché contenuto nel titolo dell'opera citata dalla domanda, incluse le esplicite dichiarazioni di informazione non disponibile. Il criterio lessicale resta pertanto una validazione incrociata e non una metrica sostitutiva; la Sezione 8.2 discute i limiti residui.

Parallelamente sono stati acquisiti i seguenti parametri:

- **Latenze di fase:** segmentazione del tempo di risposta (definito in millisecondi) tra la fase di retrieval (acquisizione delle evidenze dal database vettoriale o a grafo, inclusiva del tempo di embedding o generazione query) e la fase di generation (inferenza finale per la sintesi della risposta).
- **Metriche di Payload:** quantificazione del volume informativo recuperato. Per il RAG vettoriale vengono conteggiati i chunk; per le pipeline Text-to-Query viene registrato il numero di righe restituite; mentre per il paradigma Text-to-Expansion viene tracciato il conteggio degli archi estratti nei sottografi.

A garanzia della riproducibilità, la Tabella 7.1 riepiloga l'ambiente di esecuzione e i parametri operativi dell'intera campagna sperimentale.

7.2 Discussione del Trade-Off Vettoriale vs Grafo Semantico

I dati sintetizzati nella Tabella 7.2 indicano una divergenza di prestazioni fra il recupero documentale e quello strutturato. La baseline vettoriale adottata si attesta a un'accuratezza globale del 53,0% con il modello di riferimento da 26B, scendendo al 45,3% con E4B e al 51,0% con E2B; le pipeline su grafo *Graph_Query_V1* e *Graph_Expand_V1* raggiungono un massimo del 93,0%.

L'analisi stratificata per livello di difficoltà (Tabella 7.3) circoscrive l'origine del divario. La baseline vettoriale mantiene una tenuta accettabile sui quesiti *easy*, fra il 63,3% e il 71,7% a seconda del modello, mentre degrada

Componente	Configurazione
<i>Ambiente di esecuzione</i>	
Nodo di inferenza	NVIDIA DGX Spark: superchip GB10 Grace Blackwell, GPU Blackwell e CPU Grace a 20 core Arm, 128 GB di memoria unificata LPDDR5X condivisa fra CPU e GPU (nessuna VRAM dedicata separata)
Motore di serving	vLLM [22], immagine <code>nvcr.io/nvidia/vllm:26.05-py3</code> ; endpoint OpenAI-compatibili distinti per generazione ed embedding; finestra di contesto servita di 65.536 token
Nodo di orchestrazione e DB	Intel Core i7-13700H (14 core fisici, 20 logici), 16 GB RAM
Sistema operativo	Windows 11 (build 10.0.26200)
Runtime	Python 3.12.10; servizi containerizzati via Docker Compose
<i>Sistemi di persistenza</i>	
Neo4j	5.26.28 Community, heap JVM max 1 GB, plugin APOC, driver Bolt
Oxigraph	0.5.9, processo server, endpoint SPARQL su HTTP
PostgreSQL	16.14 con estensione <i>pgvector</i>
<i>Corpus e recupero vettoriale</i>	
Corpus	923 articoli Wikipedia (en), 17,2 M caratteri
Segmentazione	18.741 frammenti; target 1.200 char, overlap 200, minimo 200
Embedding	<i>jina-embeddings-v5-text-nano-retrieval</i> (239M par.), 768 dim.
Prefissi di istruzione	<i>query</i> : per le domande, <i>passage</i> : per i frammenti
Ricerca	<i>cosine similarity</i> , <i>top-k</i> = 10, nessun re-ranking
<i>Pipeline a grafo</i>	
Text-to-Query	singola query per quesito; fino a 2 ritentativi solo su errore
Temperature dei ritentativi	0,0 (primo tentativo) poi 0,2 e 0,4
Esecuzione delle query	server MCP locale (<i>stdio</i>), sola lettura, invocato dal framework
Text-to-Expansion	max 3 hop, 4 seed, 6 relazioni, 200 nodi, 300 archi
<i>Modelli e valutazione</i>	
Modelli generativi	<i>gemma-4-26B-A4B-it</i> FP8 (25,2B tot., 3,8B attivi); <i>gemma-4-E4B-it</i> (4,5B eff.); <i>gemma-4-E2B-it</i> (2,3B eff.)
Decodifica	temperatura 0,0, seed fissato, max 1.024 token in uscita
Budget di contesto	24.000 caratteri, identico per tutte le pipeline
Giudice	medesimo modello della sessione, verdetto binario a token singolo
Repliche	3 esecuzioni complete per configurazione

Tabella 7.1: Configurazione sperimentale della campagna RAG vs GraphRAG.

marcatamente su quelli *hard*, fra il 18,2% e il 27,3%. Una lettura coerente con il disegno dell'esperimento chiama in causa la frammentazione dell'evidenza: nei quesiti *multi-hop* i fatti risolutivi risiedono in voci Wikipedia distinte, separatamente per opera, autore e localizzazione, e la voce dell'entità-risposta è per costruzione esclusa dal corpus recuperabile. La ricerca per similarità, che seleziona i frammenti singolarmente più affini alla domanda, non dispone di un meccanismo per ricomporre la catena, poiché i frammenti intermedi non sono necessariamente quelli semanticamente più prossimi al quesito.

Alcune configurazioni GraphRAG, in particolare quelle basate sul modello di riferimento, mantengono prestazioni elevate anche all'aumentare della profondità logica: con esso *Graph_Query_V1* raggiunge il 100,0% di accuratezza nei casi *hard*, valore da leggere alla luce della numerosità ridotta della fascia (11 quesiti distinti). Il comportamento non è però generalizzabile all'intero paradigma: la medesima pipeline scende al 27,3% sugli *hard* con E4B, e *Graph_Expand_V1* passa dal 78,3% degli *easy* al 63,6% degli *hard* con E2B. La tenuta sulla profondità logica dipende quindi congiuntamente dalla strategia di recupero e dalla capacità del modello impiegato. Ciò che accomuna le configurazioni efficaci è la possibilità di selezionare le sole relazioni necessarie alla risposta anziché frammenti testuali contigui.

La disaggregazione completa evidenzia inoltre due comportamenti che il solo dato aggregato non lascia emergere. Il primo riguarda la topologia reificata in espansione: sul modello di riferimento *Graph_Expand_V2* resta competitiva sui quesiti *easy* (96,7%) ma cede su quelli *intermediate* (44,8%) e *hard* (54,5%), ossia esattamente dove il sottografo estratto è più esteso e il budget in archi si esaurisce prima. Il secondo riguarda le pipeline su RDF, il cui esito dipende dalla capacità del modello più che dalla classe di difficoltà: *Graph_Expand_RDF* affianca V1 sul modello di riferimento (98,3%, 79,3% e 90,9% nelle tre classi), mentre *Graph_Query_RDF* oscilla fra il 92,0% degli *intermediate* su E4B e il 3,4% della medesima classe su E2B, dove la generazione di SPARQL valido diventa il fattore limitante.

Sotto il profilo dell'efficienza, ciò si traduce in una maggiore densità informativa del contesto: le pipeline Text-to-Query trasmettono all'LLM fra 178 e 265 caratteri di evidenze, a fronte degli 11.171 caratteri del RAG vettoriale, con una riduzione che per *Graph_Query_V1* supera il 98%. Questa misura, tuttavia, quantifica le sole evidenze recuperate e non l'intero costo di prompting: le pipeline a grafo ricevono in aggiunta la descrizione dello schema (1.125 caratteri per V1, 1.389 per V2, 1.809 per RDF) e, nel caso di Text-to-Expansion, un prompt di pianificazione. Ricostruendo tale costo dalla dimensione dei prompt impiegati, *Graph_Query_V1* si attesta attorno ai 2.000 caratteri complessivi contro gli oltre 11.400 della baseline vettoriale: si tratta di una stima, non di una misura tracciata per singola esecuzione, e colloca il risparmio effettivo nell'ordine dell'80%. Il vantaggio resta consistente, ma va attribuito a questa specifica pipeline e non esteso indistintamente a tutte le configurazioni a grafo: *Graph_Expand_V2* occupa 3.671 caratteri di sole evidenze e *Graph_Expand_RDF* su E2B ne occupa 5.969. Il beneficio principale resta la minore esposizione del modello a testo non pertinente durante la sintesi.

La classe di difficoltà non coincide con la profondità del cammino, e le due dimensioni vanno tenute distinte per evitare letture improprie. Mentre *easy* e *intermediate* corrispondono rispettivamente a 1 e 2 hop, la classe *hard* è definita anche dalla natura del percorso e dalla distribuzione dei fatti fra le voci documentali, e raccoglie quesiti sia a 2 sia a 3 hop. Le due letture sono pertanto riportate separatamente (*validation.json*, campo *hop_breakdown*). Disaggregando per sola profondità, sul modello 26B la baseline vettoriale passa dal 72% a 1 hop al 23% a 2 hop, mentre *Graph_Query_V1* si mantiene fra il 92% e il 100% su tutte e tre le profondità. Disaggregando invece per tipologia di quesito, la baseline registra 88,9% sui quesiti *descriptive*, 57,6% sugli *one_hop*, 27,6% sui *multi_hop* e 18,2% sui *path*, a fronte di valori compresi fra 84,8% e 100% per *Graph_Query_V1*. Le due disaggregazioni concordano nell'indicare che il degrado della baseline è governato dalla necessità di ricomporre la catena fra documenti distinti, e non dalla sola profondità nominale del cammino.

Modello	Pipeline	Accuratezza	Latenza (ms)	Evidenza Media	Contesto (ch)	Errori (Media)
Gemma 4 26B-A4B	rag	53,0%	1033 ± 33	10.0 chunk	11.170	0,0
	graph_query_v1	93,0%	2357 ± 42	1.3 row	185	0,0
	graph_query_v2	92,0%	2536 ± 27	1.3 row	210	0,0
	graph_query_rdf	69,0%	2718 ± 22	1.2 row	265	0,0
	graph_expand_v1	93,0%	1548 ± 1	13.8 edge	1.158	0,0
	graph_expand_v2	77,0%	2327 ± 1	51.0 edge	3.671	0,0
	graph_expand_rdf	92,0%	1220 ± 3	14.4 edge	1.194	0,0
Gemma 4 E4B	rag	45,3% $\pm 0,6$	1369 ± 576	10.0 chunk	11.170	0,0
	graph_query_v1	65,7% $\pm 0,6$	4420 ± 1278	1.1 row	178	2,0
	graph_query_v2	60,7% $\pm 0,6$	3864 ± 158	1.0 row	188 ± 1	2,0
	graph_query_rdf	58,7% $\pm 0,6$	10227 ± 20	0.9 row	242	27,3 $\pm 0,6$
	graph_expand_v1	89,0%	2099 ± 69	17.1 edge	1.395	0,0
	graph_expand_v2	79,0%	2396 ± 33	17.9 edge	1.371	0,0
	graph_expand_rdf	86,0%	2238 ± 22	15.9 edge	1.306	0,0
Gemma 4 E2B	rag	51,0%	712 ± 7	10.0 chunk	11.170	0,0
	graph_query_v1	67,3% $\pm 0,6$	2335 ± 72	1.2 row	183	0,0
	graph_query_v2	68,0%	2468 ± 23	2.1 row	220	0,0
	graph_query_rdf	17,0%	2854 ± 1	1.1 row	249	6,0
	graph_expand_v1	71,0%	1760 ± 2	25.1 edge	1.927	0,0
	graph_expand_v2	61,0%	1846 ± 2	34.4 edge	2.495	0,0
	graph_expand_rdf	65,0%	1996 ± 3	84.6 edge	5.969	0,0

Tabella 7.2: Confronto prestazioni complessive per modello e pipeline, medie su 3 repliche complete. La deviazione standard fra repliche è riportata solo dove non nulla: a temperatura 0 e seed fissato varia in modo apprezzabile la sola latenza, sensibile al carico della macchina di test. In grassetto il valore migliore di ciascuna metrica entro il blocco di ogni modello; in **blu** il migliore dell'intera colonna, ossia fra tutti i modelli e tutte le pipeline. Sono considerate migliori l'accuratezza più alta e i valori più bassi di latenza e contesto; in caso di parità sono evidenziate tutte le configurazioni a pari merito. L'evidenza media non è evidenziata poiché espressa in unità eterogenee, né la colonna degli errori, nulla per la maggior parte delle configurazioni.

Modello	Pipeline	Easy (1 hop)	Intermediate (2 hop)	Hard (cammini)
Gemma 4 26B-A4B	rag	71,7%	27,6%	18,2%
	graph_query_v1	91,7%	93,1%	100,0%
	graph_query_v2	91,7%	89,7%	100,0%
	graph_query_rdf	66,7%	86,2%	36,4%
	graph_expand_v1	100,0%	79,3%	90,9%
	graph_expand_v2	96,7%	44,8%	54,5%
	graph_expand_rdf	98,3%	79,3%	90,9%
Gemma 4 E4B	rag	63,3%	18,4% $\pm 2,0$	18,2%
	graph_query_v1	69,4% $\pm 1,0$	72,4%	27,3%
	graph_query_v2	73,3%	57,5% $\pm 2,0$	0,0%
	graph_query_rdf	41,7%	92,0% $\pm 2,0$	63,6%
	graph_expand_v1	96,7%	72,4%	90,9%
	graph_expand_v2	88,3%	69,0%	54,5%
	graph_expand_rdf	90,0%	75,9%	90,9%
Gemma 4 E2B	rag	70,0%	20,7%	27,3%
	graph_query_v1	70,6% $\pm 1,0$	51,7%	90,9%
	graph_query_v2	75,0%	55,2%	63,6%
	graph_query_rdf	23,3%	3,4%	18,2%
	graph_expand_v1	78,3%	58,6%	63,6%
	graph_expand_v2	71,7%	44,8%	45,5%
	graph_expand_rdf	78,3%	37,9%	63,6%

Tabella 7.3: Accuratezza dettagliata per classe di difficoltà sui tre modelli, medie su 3 repliche complete; la deviazione standard è riportata solo dove non nulla. In grassetto l'accuratezza più alta di ciascuna classe entro il blocco di ogni modello; in **blu** la più alta dell'intera colonna, ossia fra tutti i modelli e tutte le pipeline. In caso di parità sono evidenziate tutte le configurazioni a pari merito.

7.3 Confronto delle Strategie a Grafo: Query Generation vs Graph Expansion

Il confronto interno ai paradigmi GraphRAG, Text-to-Query e Text-to-Expansion, evidenzia divergenze in termini di robustezza sistemistica e tolleranza ai guasti.

La generazione di query strutturate (Cypher/SPARQL) si è rivelata dipendente dalla capacità della variante impiegata. Come documentato dal conteggio degli errori (Tabella 7.2), le varianti meno capaci faticano a mantenere la correttezza sintattica e l'aderenza allo schema del database. La variante E4B, ad esempio, produce in media 27,3 query non valide per replica completa del benchmark, ossia su 100 quesiti, nella pipeline *Graph_Query_RDF*, con tempi che superano i 10.000 ms per effetto dei ritentativi innescati dalle query non valide; la stessa pipeline su E2B scende al 17,0% di accuratezza. Il recupero fondato sulla generazione di query native risulta quindi poco affidabile quando il modello disponibile non è sufficientemente capace, tanto più a fronte di uno schema reificato o di una sintassi verbosa come SPARQL.

L'approccio *Graph Expansion* (V1/RDF) si è invece rivelato più resiliente, essendo basato sulla separazione tra pianificazione logica ed esecuzione fisica. Delegando al modello esclusivamente la formulazione di parametri di navigazione (in formato JSON) e affidando al motore Python l'estrazione fisica dei record, il sistema garantisce latenze inferiori e stabili e abbatte a zero gli errori di formattazione anche nei modelli più piccoli (4B e 2B). Sul modello 26B la media è di 1.548 ms per l'espansione su V1 contro 2.357 ms per la generazione di query, con una dispersione fra le tre repliche trascurabile rispetto al divario: rispettivamente ± 1 ms (intervallo 1.547–1.549) e ± 42 ms (intervallo 2.314–2.398). Poiché le repliche sono soltanto tre, la deviazione va intesa come indicatore di

ripetibilità della singola configurazione e non come stima della varianza campionaria; il divario fra le due pipeline resta comunque di un ordine di grandezza superiore alla dispersione osservata.

Infine, il confronto fra le due topologie LPG mostra che la reificazione è associata a un calo di accuratezza in tutte le configurazioni *graph_expand*: dal 93% al 77% per il modello 26B, dall'89% al 79% per E4B e dal 71% al 61% per E2B.

Il calo non è attribuibile alla saturazione del budget testuale. Il limite disponibile è di 24.000 caratteri, mentre le configurazioni V2 ne occupano 3.671 (26B), 1.371 (E4B) e 2.495 (E2B): nessuna si avvicina alla soglia. Il caso di E4B depone in senso contrario, poiché V2 impiega 1.371 caratteri contro i 1.395 di V1, un contesto lievemente più piccolo, a fronte di un'accuratezza che scende comunque dall'89% al 79%.

Va invece isolato un secondo fattore, di natura contabile. I limiti dell'espansione (*EXPAND_MAX_NODES* = 200, *EXPAND_MAX_EDGES* = 300) contano nodi e archi fisici e non fatti logici: poiché V2 rappresenta una relazione reificata con due archi anziché uno, esaurisce il budget più rapidamente a parità di informazione trasmessa. La verifica condotta a posteriori sulle tracce persistite (*validate_results.py*) conferma l'asimmetria: il limite sugli archi è raggiunto in 9 casi su 900 per V1 contro 69 per V2. Depurando i quesiti troncati, l'accuratezza di V2 sale dal 72,3% al 76,2% mentre V1 resta invariata, e sul modello 26B il divario si riduce da 16 a 8 punti percentuali. Circa metà dello scarto osservato è dunque un artefatto della metrica di budget, mentre la metà residua persiste anche fra i soli quesiti non troncati.

Per tale quota residua resta come ipotesi non verificata che il costo sia di natura strutturale: attraversando i nodi statement, il sottografo descrive lo stesso fatto in due archi anziché in uno, e il modello deve ricomporre la relazione logica dai suoi frammenti. L'aumento della densità media degli archi in V2 (da 13,8 a 51,0 sul modello 26B) è coerente con questa lettura, ma il disegno adottato non la isola: verificarla richiederebbe di esprimere il budget in relazioni logiche anziché in archi fisici, variando la sola forma di serializzazione a parità di fatti trasmessi.

8. Conclusioni e Sviluppi Futuri

Si sintetizzano di seguito i risultati metodologici e prestazionali del progetto, i limiti emersi durante la sperimentazione e le possibili direzioni di sviluppo per le architetture ibride nel dominio del cultural heritage.

8.1 Sintesi dei Risultati e Contributi Metodologici

Il progetto ha validato la fattibilità di una pipeline deterministica per la trasformazione di dati enciclopedici in knowledge graph per l'integrazione con modelli generativi (LLM). Attraverso l'applicazione di query *CONSTRUCT* mirate, è stato generato l'artefatto *ArtGraph RDF* (18.958 triple), caratterizzato da una riduzione del rumore semantico e da una normalizzazione delle gerarchie geografiche e temporali. Sotto il profilo prestazionale, il confronto tra i modelli LPG (V1 e V2) ha quantificato l'impatto della reificazione delle relazioni sull'attraversamento del grafo. Nel benchmark di recupero informativo, le pipeline strutturate hanno superato la baseline vettoriale adottata. I dati sperimentali indicano che:

- Sui quesiti *hard*, il paradigma Text-to-Expansion su V1 ha raggiunto il 90,9% di accuratezza contro il 18,2% della baseline RAG vettoriale, con il modello di riferimento da 26B; sull'intero set di quesiti il confronto è 93,0% contro 53,0%.
- Le pipeline a grafo trasmettono al modello un volume di evidenze nettamente inferiore: 185 caratteri per Text-to-Query su V1 contro gli 11.171 del RAG vettoriale. Contabilizzando l'intero costo di prompting (schema, pianificazione ed evidenze), la stima del risparmio complessivo è dell'ordine dell'80%.
- La separazione fra pianificazione logica ed esecuzione fisica adottata dal paradigma Text-to-Expansion azzerà gli errori di formattazione su tutte e tre le varianti di modello, laddove la generazione diretta di query native produce fino a 27,3 interrogazioni non valide su 100 quesiti con la variante E4B su SPARQL. Il vantaggio è tanto più marcato quanto minore è la capacità del modello impiegato, il che rende l'approccio adatto a contesti di inferenza locale su hardware limitato.

Al di là dei valori misurati, il lavoro consegna alcuni risultati di metodo che ne costituiscono il contributo più stabile, poiché non dipendono dalla specifica campagna sperimentale:

- **Equivalenza verificata rispetto alla sorgente canonica.** Entrambe le topologie LPG restituiscono, sulle 35 query del workload, result set coincidenti con quelli prodotti dalla sorgente RDF (35/35 sia per V1 sia per V2). La trasformazione da RDF a property graph risulta quindi verificata sul workload e non semplicemente asserita, pur restando un'equivalenza rispetto alle relazioni logiche proiettate e non alla semantica integrale degli statement originali.
- **Misure separate da artefatti di misurazione.** Due fenomeni inizialmente attribuibili alle topologie si sono rivelati, all'analisi, proprietà dello strumento di misura: l'instabilità degli accessi logici fra cicli di ricarica-mento, che ha imposto di ragionare su rapporti fra configurazioni anziché su valori assoluti, e il troncamento

dell'espansione, che spiega circa metà del divario V1–V2 in accuratezza. In entrambi i casi l'artefatto è stato quantificato e scorporato dal risultato riportato.

- **Verdetti di correttezza sottoposti a controllo indipendente.** La valutazione automatica non è stata assunta come affidabile ma verificata contro un criterio deterministico esterno al modello, con un accordo del 96,2% su 6.300 risposte. La persistenza integrale delle tracce, che include risposte, riferimenti e verdeti, rende ogni decisione ispezionabile e consente di ri-arbitrare l'intera campagna senza rieseguire le pipeline.

Questi risultati valgono per il benchmark costruito in questo progetto (domande derivate dai cammini del grafo, corpus Wikipedia allineato e baseline vettoriale non ottimizzata) e non costituiscono una comparazione generale fra i paradigmi RAG e GraphRAG. La Sezione 8.2 ne circoscrive la portata.

8.2 Limitazioni

I risultati ottenuti vanno letti entro un perimetro preciso. Si elencano di seguito le principali limitazioni del lavoro, distinte per ambito.

Costruzione del benchmark di recupero.

- **Bias di costruzione delle domande:** i quesiti sono derivati per ispezione deterministica dei cammini di *artgraph.ttl*, ossia dalla medesima rappresentazione su cui operano le pipeline GraphRAG. Una domanda esprimibile come cammino nel grafo è per costruzione risolvibile da una traversata, mentre lo è solo indirettamente da un recupero testuale. Il confronto va quindi inteso come *benchmark multi-hop derivato da grafo* e non come valutazione neutrale dei due paradigmi.
- **Esclusione della voce-risposta:** dal corpus recuperabile è esclusa sistematicamente la voce dell'entità-risposta, così da imporre un effettivo concatenamento fra documenti. La scelta è funzionale a testare il comportamento multi-hop, ma penalizza deliberatamente la baseline vettoriale rispetto a uno scenario d'uso reale, in cui quella voce sarebbe disponibile.
- **Numerosità delle fasce di difficoltà:** la fascia *hard* comprende 11 quesiti distinti. Le percentuali che la riguardano, inclusi i valori pari al 100%, poggiano su una base campionaria ridotta e vanno considerate indicative.

Baseline e strumenti di misura.

- **Semplicità della baseline vettoriale:** il RAG adottato è una configurazione densa essenziale: top-10 per *cosine similarity*, senza re-ranking, decomposizione della domanda, interrogazione multipla o recupero iterativo. È una scelta deliberata, volta a isolare la modalità con cui l'evidenza raggiunge il modello; ne consegue però che i risultati riguardano questa specifica baseline e non il paradigma RAG in generale. Tecniche più evolute, dalla riformulazione della domanda tramite documenti ipotetici [23] al recupero iterativo, potrebbero ridurre il divario e costituiscono baseline da valutare.
- **Valutazione tramite LLM-as-a-judge:** la correttezza è stabilita da un modello e non da annotatori umani, e non è stato misurato l'accordo con un giudizio umano su campione; la letteratura documenta per i giudici automatici fenomeni di *verbosity bias* e *self-preference* [16]. Nel disegno adottato il giudice coincide inoltre con la variante che genera la risposta, per cui le accuratezze dei tre modelli non sono stabilite da un metro comune: un confronto diretto fra varianti (il 53,0% del RAG su 26B contro il 45,3% su E4B) può riflettere in

parte una diversa severità del valutatore. Il confronto fra pipeline *a parità di modello*, oggetto primario del benchmark, non è invece interessato dalla distorsione, poiché entro una stessa sessione tutte le pipeline sono valutate dal medesimo giudice. La validazione lessicale incrociata (Sezione 7.1) circoscrive il fenomeno senza eliminarlo; poiché risposte e riferimenti sono integralmente persistiti, una ri-arbitratura con un unico giudice fisso resta possibile senza rieseguire le pipeline.

Costruzione del grafo e benchmark dei DBMS.

- **Circolarità fra scope e workload:** i confini del grafo sono definiti a partire dalle necessità informative delle 35 query, che vengono poi impiegate per verificarne l'espressività. L'impostazione è coerente con un benchmark *workload-driven*, ma il superamento delle 35 query non costituisce una validazione generale del Knowledge Graph.
- **Portata dell'equivalenza fra RDF e LPG:** la verifica di equivalenza è condotta sui risultati delle 35 query e sulle relazioni logiche proiettate, non sulla semantica integrale degli statement RDF. La sorgente conserva esplicitamente la molteplicità degli statement tramite 1.332 nodi dedicati, che si riducono a 789 coppie distinte, mentre V2 ne reifica 752 deduplicate; inoltre alcune query SPARQL riducono valori multipli tramite MIN, ad esempio su periodi e numeri d'inventario. Si tratta pertanto di un'equivalenza rispetto al workload e alle relazioni logiche proiettate, e non di una conservazione integrale dell'informazione degli statement originali.
- **Confronto fra motori non isolato:** Neo4j e Oxigraph sono interrogati attraverso protocolli differenti (Bolt su TCP contro SPARQL su HTTP) e la loro ingestione comporta operazioni non equivalenti. Il divario osservato è pertanto proprio di questa implementazione e di questo workload, e non dimostra una superiorità intrinseca di un modello dei dati sull'altro.

Fragilità del paradigma Text-to-Query. La generazione di codice Cypher/SPARQL si è rivelata sensibile alla capacità del modello impiegato: le varianti minori hanno prodotto errori sintattici e violazioni di schema in misura tale da comprometterne l'affidabilità operativa, con un massimo di 27,3 errori medi per esecuzione nella configurazione RDF su E4B.

8.3 Prospettive di Ricerca ed Evoluzioni Future

L'architettura proposta in questo progetto pone le basi per evoluzioni orientate alla convergenza tra dati strutturati e non strutturati:

- **Hybrid RAG:** Sviluppo di strategie di recupero capaci di integrare indici vettoriali su testi estesi con query topologiche, permettendo analisi qualitative e quantitative (metadati strutturati) simultanee.
- **Integrazione della Provenienza (Data Lineage):** Mappatura sistematica dei qualificatori di origine e dei riferimenti bibliografici all'interno dello schema Neo4j, al fine di poter effettuare interrogazioni sulla tracciabilità storica dei dati.
- **Estensione Multimodale:** Integrazione di modelli *Vision-Language* (VLM) per includere descrizioni visive estratte direttamente dalle immagini dei dipinti come nodi semantici nel grafo.

Bibliografia

- [1] P. Lewis et al., «Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,» in *Advances in Neural Information Processing Systems*, 2020.
- [2] Z. Yang et al., «HotpotQA: A Dataset for Diverse, Explainable Multi-hop Question Answering,» in *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2018.
- [3] D. Edge et al., *From Local to Global: A Graph RAG Approach to Query-Focused Summarization*, arXiv:2404.16130, 2024.
- [4] M. Grüninger e M. S. Fox, «Methodology for the Design and Evaluation of Ontologies,» in *Proceedings of the IJCAI Workshop on Basic Ontological Issues in Knowledge Sharing*, 1995.
- [5] W3C, *RDF 1.1 Turtle: Terse RDF Triple Language*, <https://www.w3.org/TR/turtle/>, W3C Recommendation, 25 febbraio 2014. Consultato il 7 agosto 2026.
- [6] Wikidata, *Wikidata Documentation*, <https://www.wikidata.org/wiki/Wikidata:Introduction>, Documentazione ufficiale consultata per il modello dati di Wikidata, gli item, le proprietà, gli statement e l'accesso tramite Wikidata Query Service. Consultato il 7 luglio 2026.
- [7] Getty Research Institute, *Getty Vocabularies as Linked Open Data*, <https://www.getty.edu/research/tools/vocabularies/lod/index.html>, Consultato il 7 luglio 2026.
- [8] W3C, *RDF 1.1 Concepts and Abstract Syntax*, <https://www.w3.org/TR/rdf11-concepts/>, W3C Recommendation, 25 febbraio 2014. Consultato il 7 agosto 2026.
- [9] Neo4j, *Neo4j Documentation*, <https://neo4j.com/docs/>, Documentazione ufficiale consultata per il modello property graph, Cypher, vincoli, indici e full-text indexes. Consultato il 7 luglio 2026.
- [10] Jina AI, *jina-embeddings-v5-text-nano-retrieval*, <https://huggingface.co/jinaai/jina-embeddings-v5-text-nano-retrieval>, 239M parametri, 768 dimensioni con supporto Matryoshka, finestra di 8192 token. Consultato il 7 agosto 2026.
- [11] V. Karpukhin et al., «Dense Passage Retrieval for Open-Domain Question Answering,» in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2020.
- [12] pgvector, *pgvector: Open-source vector similarity search for Postgres*, <https://github.com/pgvector/pgvector>, Consultato il 7 agosto 2026.
- [13] W3C, *SPARQL 1.1 Query Language*, <https://www.w3.org/TR/sparql11-query/>, W3C Recommendation, 21 marzo 2013. Consultato il 7 agosto 2026.
- [14] Oxigraph, *Oxigraph: a SPARQL database and RDF toolkit*, <https://github.com/oxigraph/oxigraph>, Consultato il 7 agosto 2026.

- [15] Model Context Protocol, *Model Context Protocol Specification*, <https://modelcontextprotocol.io/>, Protocollo creato da Anthropic e donato nel dicembre 2025 alla Agentic AI Foundation (Linux Foundation). Consultato il 7 agosto 2026.
- [16] L. Zheng et al., «Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena,» in *Advances in Neural Information Processing Systems 36 (NeurIPS), Datasets and Benchmarks Track*, 2023.
- [17] NVIDIA, *NVIDIA DGX Spark*, <https://www.nvidia.com/en-us/products/workstations/dgx-spark/>, Superchip GB10 Grace Blackwell, 128 GB di memoria unificata LPDDR5X. Consultato il 7 agosto 2026.
- [18] Google DeepMind, *gemma-4-26B-A4B-it*, <https://huggingface.co/google/gemma-4-26B-A4B-it>, Variante Mixture-of-Experts con 25,2B parametri totali e 3,8B attivi per token. Consultato il 7 agosto 2026.
- [19] Red Hat AI, *gemma-4-26B-A4B-it-FP8-dynamic*, <https://huggingface.co/RedHatAI/gemma-4-26B-A4B-it-FP8-dynamic>, Quantizzazione FP8 dinamica di *google/gemma-4-26B-A4B-it* tramite LLM Compressor. Consultato il 7 agosto 2026.
- [20] Unsloth AI, *gemma-4-E4B-it*, <https://huggingface.co/unsloth/gemma-4-E4B-it>, Redistribuzione dei pesi di *google/gemma-4-E4B-it* effettivamente impiegata; 8B parametri totali, 4,5B effettivi grazie alle Per-Layer Embeddings. Consultato il 7 agosto 2026.
- [21] Unsloth AI, *gemma-4-E2B-it*, <https://huggingface.co/unsloth/gemma-4-E2B-it>, Redistribuzione dei pesi di *google/gemma-4-E2B-it* effettivamente impiegata; 5,1B parametri totali, 2,3B effettivi grazie alle Per-Layer Embeddings. Consultato il 7 agosto 2026.
- [22] vLLM, *vLLM: Easy, Fast, and Cheap LLM Serving*, <https://github.com/vllm-project/vllm>, Motore di serving impiegato per i modelli generativi e di embedding. Consultato il 7 agosto 2026.
- [23] L. Gao, X. Ma, J. Lin e J. Callan, «Precise Zero-Shot Dense Retrieval without Relevance Labels,» in *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL)*, 2023.